

Optimisation pour l'IA

Cours1 - Optimisation continue et linéaire

MS IA DE CONFIANCE · CENTRALESUPÉLEC

Nadjib LAZAAR

Ing - Phd - HDR - Professor

Paris-Saclay University - LISN - DataIA Institute

<https://perso.lisn.upsaclay.fr/lazaar/>

24/09/2026

Optimisation pour l'IA

Les trois séances

- **S1: Optimisation continue et linéaire**
 - **Modéliser** : variables, objectif, contraintes
 - **Résoudre** : descente de gradient, simplexe
- **S2: Optimisation discrète + Programmation par contraintes**
 - Formulations entières / 0-1
 - Séparation-évaluation, coupes de Gomory
 - CP-SAT : un autre paradigme
- **S3: Problèmes de transport et graphes**
 - Simplexe de transport
 - Plus courts chemins, ACM, flot à coût min
 - Soutenance du projet **ExedBike**

Agenda de la Séance 1

4 heures

- **Cours - Modélisation**

- Traduire un problème réel en formulation mathématique : variables, objectif, contraintes

- **TD1 - Exercices papier (modélisation)**

- **Cours - Résolution**

- Descente de gradient (continu), simplexe (linéaire), et ouverture vers les problèmes combinatoires difficiles

- **TD1 - Exercices papier (résolution)**

- **TP1 — Atelier Python & Excel**

L'optimisation est partout

Bien au-delà du machine learning

- **Planifier des tournées de livraison**
= choisir l'ordre des arrêts qui minimise la distance parcourue
- **Répartir une production entre plusieurs lignes**
= maximiser la marge sous contraintes de ressources
- **Régler finement les paramètres d'un modèle numérique**
= minimiser l'écart aux observations (le machine learning n'est qu'un cas parmi d'autres)
- **Décider quels équipements construire, et dans quel ordre**
= choisir parmi un nombre gigantesque de combinaisons possibles

.

L'optimisation est partout

Bien au-delà du machine learning

- Planifier des tournées de livraison

= choisir l'ordre des arrêts qui minimise la distance parcourue

-

La compétence la plus importante : transformer une question réelle en problème mathématique — puis reconnaître à quelle famille il appartient, car cela détermine si (et comment) on peut le résoudre

n'est qu'un cas parmi d'autres)

- Décider quels équipements construire, et dans quel ordre

= choisir parmi un nombre gigantesque de combinaisons possibles

-

Deux grandes familles de problèmes

Domaine continu / Domaine discret

Variables continues

x peut prendre n'importe quelle valeur réelle

- Exemple : un prix, un poids, un réglage à ajuster
- Se résout en général efficacement

Variables discrètes / combinatoires

x doit prendre des valeurs entières ou binaires

- Exemple : ouvrir ou non une usine, choisir un ordre de livraison
- Peut devenir extrêmement difficile à résoudre exactement

Deux grandes familles de problèmes

Domaine continu / Domaine discret

Variables continues

x peut prendre n'importe quelle valeur réelle

- Exemple : un prix, un poids, un réglage à ajuster
- Se résout en général efficacement

Variables discrètes / combinatoires

x doit prendre des valeurs entières ou binaires

- Exemple : ouvrir ou non une usine, choisir un ordre de livraison
- Peut devenir extrêmement difficile à résoudre exactement

La programmation linéaire est le pont entre les deux : ses variables sont continues, mais sa structure géométrique (sommets, contraintes) est la même qu'on retrouvera pour attaquer les problèmes combinatoires

Qu'est-ce qu'"optimiser", concrètement ?

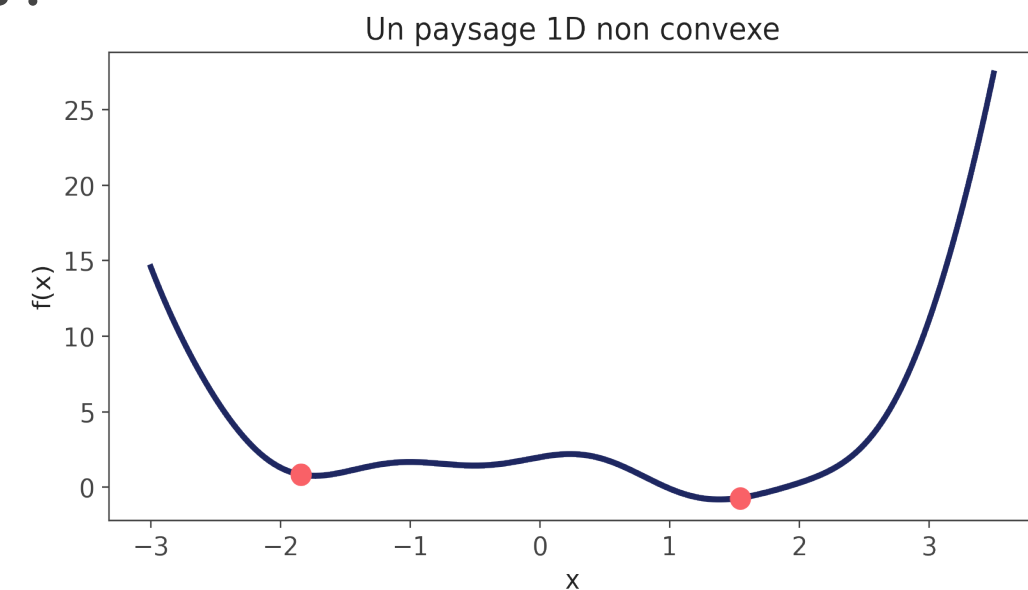
Intuition d'abord

- Imaginez chercher le point le plus bas d'une vallée, les yeux bandés
- Vous ne voyez pas toute la vallée : la fonction à optimiser est souvent trop complexe pour être visualisée d'un coup
- Vous ne sentez que la pente sous vos pieds — c'est votre seule information locale : le gradient
- **Descendre dans le sens le plus raide, un petit pas à la fois !**

Qu'est-ce qu'"optimiser", concrètement ?

Intuition d'abord

- Imaginez chercher le point le plus bas d'une vallée, les yeux bandés
- Vous ne voyez pas toute la vallée : la fonction à optimiser est souvent trop complexe pour être visualisée d'un coup
- Vous ne sentez que la pente sous vos pieds — c'est votre seule information locale : le gradient
- **Descendre dans le sens le plus raide, un petit pas à la fois !**



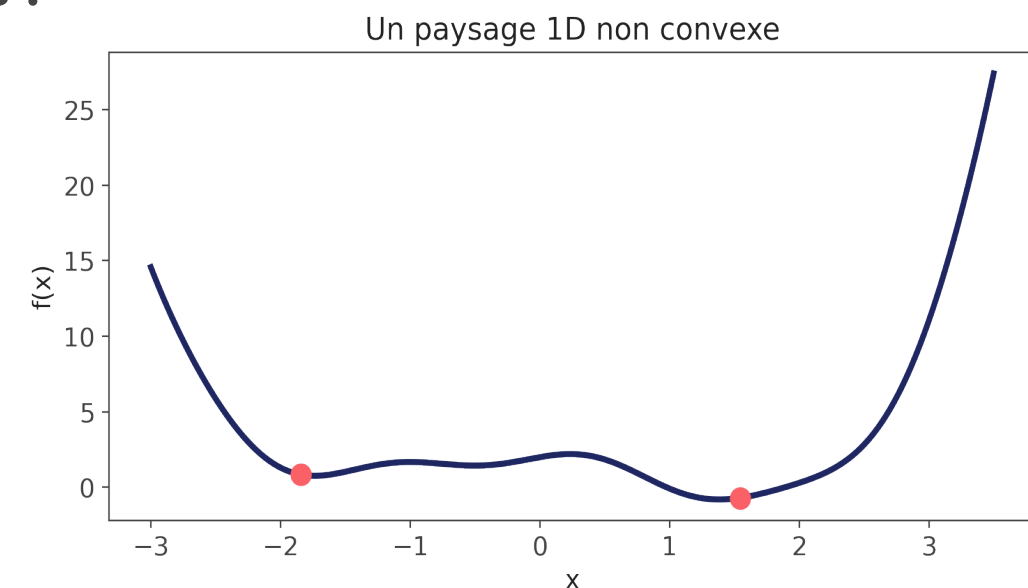
Qu'est-ce qu'"optimiser", concrètement ?

Intuition d'abord

- Imaginez chercher le point le plus bas d'une vallée, les yeux bandés
- Vous ne voyez pas toute la vallée : la fonction à optimiser est souvent trop complexe pour être visualisée d'un coup
- Vous ne sentez que la pente sous vos pieds — c'est votre seule information locale : le gradient
- **Descendre dans le sens le plus raide, un petit pas à la fois !**

Optimiser, c'est toujours répondre à trois questions :

1. que cherche-t-on à régler (les variables) ?
2. que veut-on rendre le plus petit ou le plus grand possible (l'objectif) ?
3. quelles limites doit-on respecter (les contraintes) ?



Le problème d'optimisation générique

Formalisme

minimiser $f_0(x)$
Sous contraintes $f_i(x) \leq b_i$

- $x \in R^n$: les variables (ce qu'on cherche à régler)
- f_0 : l'objectif (ce qu'on minimise, ou maximise en changeant le signe)
- Les $f_i(x) \leq b_i$: les contraintes, les limites à respecter
- x^* est optimal s'il a le plus petit f_0 parmi tous les x admissibles

Le problème d'optimisation générique

Formalisme

minimiser $f_0(x)$
Sous contraintes $f_i(x) \leq b_i$

- $x \in R^n$: les variables (ce qu'on cherche à régler)
- f_0 : l'objectif (ce qu'on minimise, ou maximise en changeant le signe)
- Les $f_i(x) \leq b_i$: les contraintes, les limites à respecter
- x^* est optimal s'il a le plus petit f_0 parmi tous les x admissibles

EXEMPLE CONCRET — ExedBike



- x = nombre de vélos Urban et Cargo à produire
- $f_0(x) = -$ marge totale (on minimise l'opposé pour "maximiser" la marge)
- $f_i(x) \leq b_i$ = limites de main-d'œuvre, d'aluminium, de temps machine disponibles
- x^* = **le plan de production qui rapporte le plus, sans dépasser aucune ressource**

Un autre style de programmation

Déclaratif plutôt qu'impératif

Impératif (procédural)

```
w = w0  
  
for k in range(K):  
    w = w - alpha * grad(w)
```

*Vous décrivez le calcul à effectuer, étape par étape
=> le "comment"*

Un autre style de programmation

Déclaratif plutôt qu'impératif

Impératif (procédural)

```
w = w0  
  
for k in range(K):  
    w = w - alpha * grad(w)
```

***`Vous décrivez le calcul à effectuer, étape par étape
=> le "comment"`***

Déclaratif

```
variables : x  
  
minimiser f0(x)  
  
s.c. fi(x) ≤ bi
```

***`Vous décrivez ce que vous voulez => le solveur trouve
le "comment"`***

Un autre style de programmation

Déclaratif plutôt qu'impératif

Impératif (procédural)

```
w = w0  
  
for k in range(K):  
    w = w - alpha * grad(w)
```

***Vous décrivez le calcul à effectuer, étape par étape
=> le "comment"***

Déclaratif

```
variables : x  
  
minimiser f0(x)  
  
s.c. fi(x) ≤ bi
```

***Vous décrivez ce que vous voulez => le solveur trouve
le "comment"***

Style déclaratif	Vous écrivez	Exemple dans ce cours
Programmation linéaire	Variables, objectif linéaire, contraintes linéaires	Le mix de production ExedBike
Programmation par contraintes (PPC)	Variables, domaines, contraintes souvent	Ordonnancement, Sudoku (Séance 2)

1

Modélisation

Avant de résoudre quoi que ce soit, il faut savoir écrire le problème :
quelles variables, quel objectif, quelles contraintes. C'est l'étape la plus
importante — et la plus souvent négligée

Exemple : ajuster une courbe de coût

Modéliser (continu)

- Un atelier a mesuré son coût de production pour plusieurs volumes : plus on produit, plus le coût unitaire baisse, jusqu'à un certain point
- On veut un modèle simple $\text{coût}(v) = a \cdot v + b$ qui "colle" au mieux à ces mesures, pour pouvoir prédire le coût à un nouveau volume
- **Variables : les paramètres du modèle, a et b (pas le volume v , qui est donné par les données)**
- **Objectif : minimiser l'écart total entre le modèle et les mesures — par exemple la somme des écarts au carré**
- **Contraintes** : ici, aucune — c'est un problème continu et libre. Nous le résoudrons par descente de gradient (en Partie 2)

Exemple : ajuster une courbe de coût

Modéliser (continu)

- Un atelier a mesuré son coût de production pour plusieurs volumes : plus on produit, plus le coût unitaire baisse, jusqu'à un certain point
- On veut un modèle simple $\text{coût}(v) = a \cdot v + b$ qui "colle" au mieux à ces mesures, pour pouvoir prédire le coût à un nouveau volume
- **Variables : les paramètres du modèle, a et b (pas le volume v , qui est donné par les données)**
- **Objectif : minimiser l'écart total entre le modèle et les mesures — par exemple la somme des écarts au carré**
- **Contraintes** : ici, aucune — c'est un problème continu et libre. Nous le résoudrons par descente de gradient (en Partie 2)

Formulation

$$\text{minimiser}(a, b) \quad \sum_k (a \cdot v_k + b - \text{cout}_k)^2$$

C'est exactement le principe des "moindres carrés" — la même idée sert aussi, plus tard, à entraîner un modèle de machine learning : ajuster des paramètres pour minimiser un écart. Ce n'est qu'une application parmi d'autres.

La convexité : pourquoi c'est si important

Reconnaitre la structure

Une fonction f est convexe si, pour deux points quelconques et tout $\lambda \in [0,1]$:

$$f(\lambda x_1 + (1 - \lambda) x_2) \leq \lambda f(x_1) + (1 - \lambda) f(x_2)$$

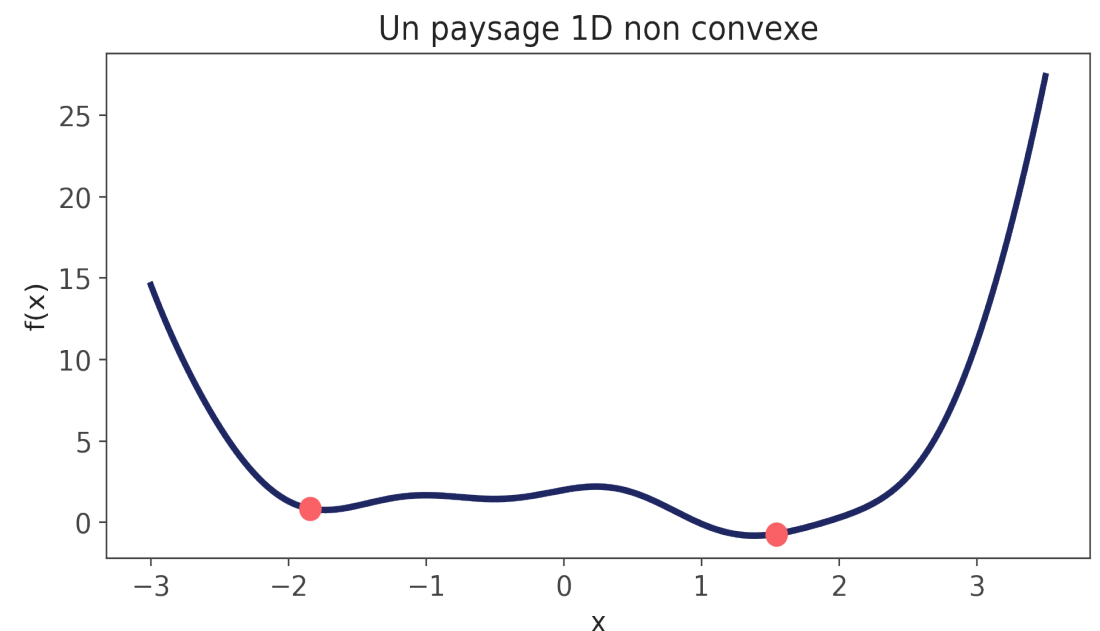
La convexité : pourquoi c'est si important

Reconnaitre la structure

Une fonction f est convexe si, pour deux points quelconques et tout $\lambda \in [0,1]$:

$$f(\lambda x_1 + (1 - \lambda) x_2) \leq \lambda f(x_1) + (1 - \lambda) f(x_2)$$

- Géométriquement : la corde entre deux points quelconques du graphe reste au-dessus du graphe
=> "la vallée n'a qu'un seul fond"
- **Si f est convexe, tout minimum local est un minimum global : c'est ce qui rend un problème réellement soluble**
- Une égalité au lieu de $\leq$ définit une fonction affine ("linéaire")
- Les problèmes non convexes n'offrent aucune telle garantie : la descente de gradient ne trouve alors qu'un **minimum local**, pas forcément le meilleur



La convexité : pourquoi c'est si important

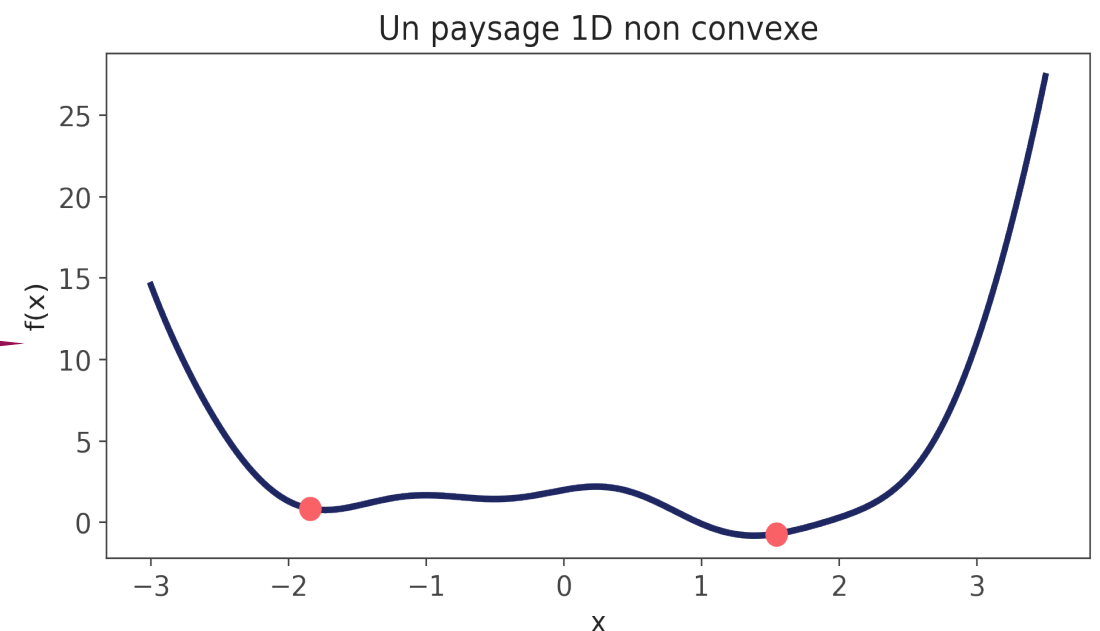
Reconnaitre la structure

Une fonction f est convexe si, pour deux points quelconques et tout $\lambda \in [0,1]$:

$$f(\lambda x_1 + (1 - \lambda) x_2) \leq \lambda f(x_1) + (1 - \lambda) f(x_2)$$

- Géométriquement : la corde entre deux points quelconques du graphe reste au-dessus du graphe => "la vallée n'a qu'un seul fond"
- **Si f est convexe, tout minimum local est un minimum global : c'est ce qui rend un problème réellement soluble**
- Une égalité au lieu de $\leq$ définit une fonction affine ("linéaire")
- Les problèmes non convexes n'offrent aucune telle garantie : la descente de gradient ne trouve alors qu'un **minimum local**, pas forcément le meilleur

Deux minima locaux, un seul global :
partie du mauvais côté, la descente de gradient trouve le mauvais creux

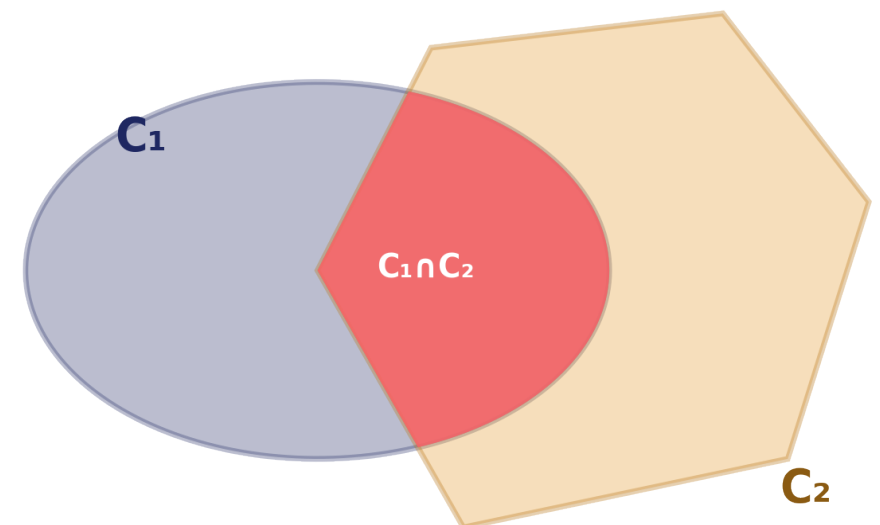


Ensembles convexes : la propriété qui fait tout tenir

Reconnaitre la structure

Ensemble C est convexe si, pour $x_1, x_2 \in C$ et $\lambda \in [0,1]$: $\lambda x_1 + (1 - \lambda) x_2 \in C$

- Même idée que pour une fonction convexe, mais pour un ensemble : le segment entre deux points quelconques de C reste entièrement dans C
- Un demi-espace $\{x : a^T x \leq b\}$ — la forme de chaque contrainte linéaire — est convexe
- **Propriété clé : l'intersection d'un nombre quelconque d'ensembles convexes est convexe**
- Preuve immédiate : si x_1, x_2 sont dans chaque C_i , le segment $[x_1, x_2]$ reste dans chaque C_i (chacun est convexe) — donc dans leur intersection.
- **Conséquence : la région admissible d'un PL, intersection de demi-espaces, est donc toujours convexe, quel que soit le nombre de contraintes**



Traduire un énoncé en programme linéaire : la méthode en 4 étapes

Modéliser (linéaire)

Traduire un énoncé en programme linéaire : la méthode en 4 étapes

Modéliser (linéaire)

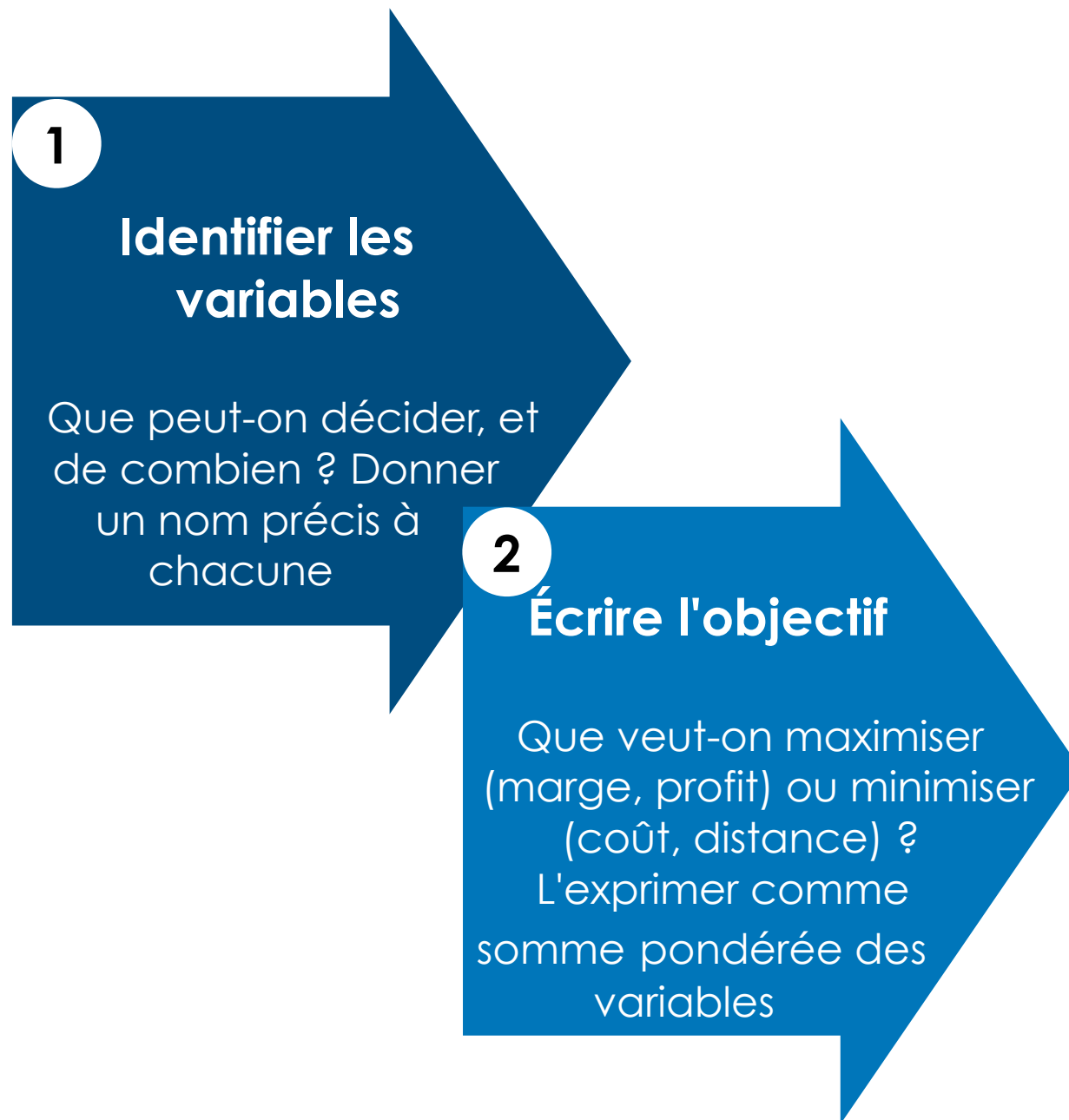
1

Identifier les variables

Que peut-on décider, et de combien ? Donner un nom précis à chacune

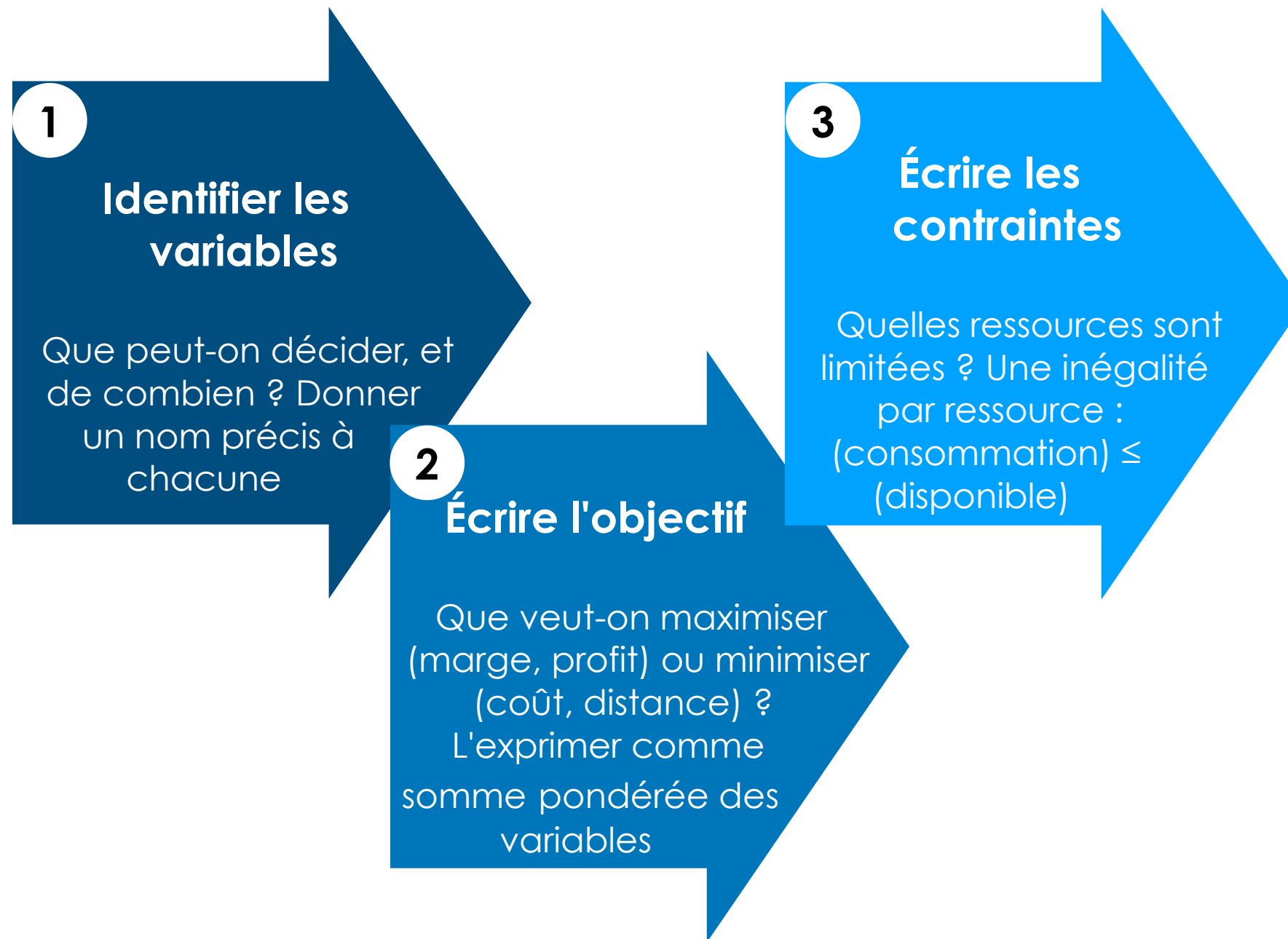
Traduire un énoncé en programme linéaire : la méthode en 4 étapes

Modéliser (linéaire)



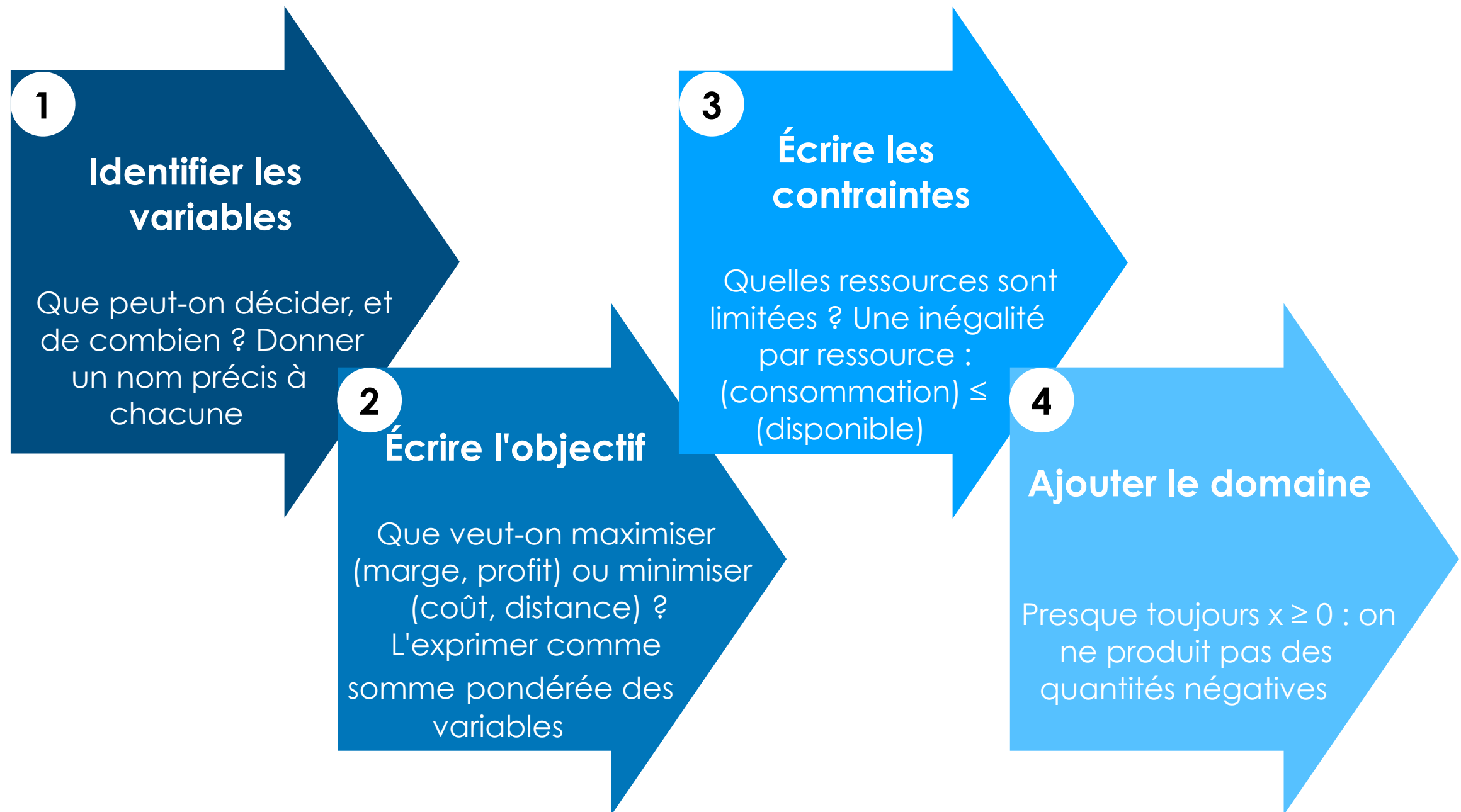
Traduire un énoncé en programme linéaire : la méthode en 4 étapes

Modéliser (linéaire)



Traduire un énoncé en programme linéaire : la méthode en 4 étapes

Modéliser (linéaire)



Modéliser un cas simple : la menuiserie

Exemple guidé

- Un artisan fabrique des tables et des chaises.
- Une table rapporte 70€ de marge et nécessite 4 planches de bois et 2h de machine.
- Une chaise rapporte 50€ et nécessite 3 planches et 1h de machine.
- L'atelier dispose de 240 planches et de 100h de machine par semaine.
- **Combien de tables et de chaises produire pour maximiser la marge ?**

Modéliser un cas simple : la menuiserie

Exemple guidé

- Un artisan fabrique des tables et des chaises.
- Une table rapporte 70€ de marge et nécessite 4 planches de bois et 2h de machine.
- Une chaise rapporte 50€ et nécessite 3 planches et 1h de machine.
- L'atelier dispose de 240 planches et de 100h de machine par semaine.
- **Combien de tables et de chaises produire pour maximiser la marge ?**

1. Variables	
2. Objectif	
3. Contraintes	
3. Domaine	

Modéliser un cas simple : la menuiserie

Exemple guidé

- Un artisan fabrique des tables et des chaises.
- Une table rapporte 70€ de marge et nécessite 4 planches de bois et 2h de machine.
- Une chaise rapporte 50€ et nécessite 3 planches et 1h de machine.
- L'atelier dispose de 240 planches et de 100h de machine par semaine.
- **Combien de tables et de chaises produire pour maximiser la marge ?**

1. Variables	
2. Objectif	
3. Contraintes	
3. Domaine	

Modéliser un cas simple : la menuiserie

Exemple guidé

- Un artisan fabrique des tables et des chaises.
- Une table rapporte 70€ de marge et nécessite 4 planches de bois et 2h de machine.
- Une chaise rapporte 50€ et nécessite 3 planches et 1h de machine.
- L'atelier dispose de 240 planches et de 100h de machine par semaine.
- **Combien de tables et de chaises produire pour maximiser la marge ?**

1. Variables	x = nombre de tables, y = nombre de chaises (par semaine)
2. Objectif	
3. Contraintes	
3. Domaine	

Modéliser un cas simple : la menuiserie

Exemple guidé

- Un artisan fabrique des tables et des chaises.
- Une table rapporte 70€ de marge et nécessite 4 planches de bois et 2h de machine.
- Une chaise rapporte 50€ et nécessite 3 planches et 1h de machine.
- L'atelier dispose de 240 planches et de 100h de machine par semaine.
- **Combien de tables et de chaises produire pour maximiser la marge ?**

1. Variables	x = nombre de tables, y = nombre de chaises (par semaine)
2. Objectif	
3. Contraintes	
3. Domaine	

Modéliser un cas simple : la menuiserie

Exemple guidé

- Un artisan fabrique des tables et des chaises.
- Une table rapporte 70€ de marge et nécessite 4 planches de bois et 2h de machine.
- Une chaise rapporte 50€ et nécessite 3 planches et 1h de machine.
- L'atelier dispose de 240 planches et de 100h de machine par semaine.
- **Combien de tables et de chaises produire pour maximiser la marge ?**

1. Variables	x = nombre de tables, y = nombre de chaises (par semaine)
2. Objectif	
3. Contraintes	
3. Domaine	

Modéliser un cas simple : la menuiserie

Exemple guidé

- Un artisan fabrique des tables et des chaises.
- Une table rapporte 70€ de marge et nécessite 4 planches de bois et 2h de machine.
- Une chaise rapporte 50€ et nécessite 3 planches et 1h de machine.
- L'atelier dispose de 240 planches et de 100h de machine par semaine.
- **Combien de tables et de chaises produire pour maximiser la marge ?**

1. Variables	x = nombre de tables, y = nombre de chaises (par semaine)
2. Objectif	maximiser $70x + 50y$
3. Contraintes	
3. Domaine	

Modéliser un cas simple : la menuiserie

Exemple guidé

- Un artisan fabrique des tables et des chaises.
- Une table rapporte 70€ de marge et nécessite 4 planches de bois et 2h de machine.
- Une chaise rapporte 50€ et nécessite 3 planches et 1h de machine.
- L'atelier dispose de 240 planches et de 100h de machine par semaine.
- **Combien de tables et de chaises produire pour maximiser la marge ?**

1. Variables	x = nombre de tables, y = nombre de chaises (par semaine)
2. Objectif	maximiser $70x + 50y$
3. Contraintes	
3. Domaine	

Modéliser un cas simple : la menuiserie

Exemple guidé

- Un artisan fabrique des tables et des chaises.
- Une table rapporte 70€ de marge et nécessite 4 planches de bois et 2h de machine.
- Une chaise rapporte 50€ et nécessite 3 planches et 1h de machine.
- L'atelier dispose de 240 planches et de 100h de machine par semaine.
- **Combien de tables et de chaises produire pour maximiser la marge ?**

1. Variables	x = nombre de tables, y = nombre de chaises (par semaine)
2. Objectif	maximiser $70x + 50y$
3. Contraintes	
3. Domaine	

Modéliser un cas simple : la menuiserie

Exemple guidé

- Un artisan fabrique des tables et des chaises.
- Une table rapporte 70€ de marge et nécessite 4 planches de bois et 2h de machine.
- Une chaise rapporte 50€ et nécessite 3 planches et 1h de machine.
- L'atelier dispose de 240 planches et de 100h de machine par semaine.
- **Combien de tables et de chaises produire pour maximiser la marge ?**

1. Variables	$x = \text{nombre de tables}, y = \text{nombre de chaises (par semaine)}$
2. Objectif	maximiser $70x + 50y$
3. Contraintes	bois : $4x + 3y \leq 240$ machine : $2x + 1y \leq 100$
3. Domaine	

Modéliser un cas simple : la menuiserie

Exemple guidé

- Un artisan fabrique des tables et des chaises.
- Une table rapporte 70€ de marge et nécessite 4 planches de bois et 2h de machine.
- Une chaise rapporte 50€ et nécessite 3 planches et 1h de machine.
- L'atelier dispose de 240 planches et de 100h de machine par semaine.
- **Combien de tables et de chaises produire pour maximiser la marge ?**

1. Variables	$x = \text{nombre de tables}, y = \text{nombre de chaises (par semaine)}$
2. Objectif	maximiser $70x + 50y$
3. Contraintes	bois : $4x + 3y \leq 240$ machine : $2x + 1y \leq 100$
3. Domaine	

Modéliser un cas simple : la menuiserie

Exemple guidé

- Un artisan fabrique des tables et des chaises.
- Une table rapporte 70€ de marge et nécessite 4 planches de bois et 2h de machine.
- Une chaise rapporte 50€ et nécessite 3 planches et 1h de machine.
- L'atelier dispose de 240 planches et de 100h de machine par semaine.
- **Combien de tables et de chaises produire pour maximiser la marge ?**

1. Variables	$x = \text{nombre de tables}, y = \text{nombre de chaises (par semaine)}$
2. Objectif	maximiser $70x + 50y$
3. Contraintes	bois : $4x + 3y \leq 240$ machine : $2x + 1y \leq 100$
3. Domaine	$x \geq 0; \quad y \geq 0$

Trois formes d'un même programme linéaire

Modéliser (linéaire)

Trois formes d'un même programme linéaire

Modéliser (linéaire)

Forme générale

min ou max $c^T x$

s.c. contraintes $\leq, \geq$ ou $=$

variables ≥ 0 ou libres

*La forme naturelle,
obtenue directement avec
la méthode en 4 étapes —
sans se soucier du format*

Trois formes d'un même programme linéaire

Modéliser (linéaire)

Forme générale

min ou max $c^T x$
s.c. contraintes $\leq, \geq$ ou $=$
variables ≥ 0 ou libres

*La forme naturelle,
obtenue directement avec
la méthode en 4 étapes —
sans se soucier du format*

Forme canonique

maximiser $c^T x$
s.c. $Ax \leq b$
 $x \geq 0$

*Format homogène
nécessaire pour construire
le dual (Partie 2) et
comparer des PL entre eux*

Trois formes d'un même programme linéaire

Modéliser (linéaire)

Forme générale

min ou max $c^T x$
s.c. contraintes $\leq, \geq$ ou $=$
variables ≥ 0 ou libres

*La forme naturelle,
obtenue directement avec
la méthode en 4 étapes —
sans se soucier du format*

Forme canonique

maximiser $c^T x$
s.c. $Ax \leq b$
 $x \geq 0$

*Format homogène
nécessaire pour construire
le dual (Partie 2) et
comparer des PL entre eux*

Forme standard

maximiser $c^T x$
s.c. $Ax = b$
 $x \geq 0$

*La forme qu'exige le
simplexe : il lui faut des
égalités pour construire le
tableau et démarrer avec
une solution de base
immédiate*

Trois formes d'un même programme linéaire

Modéliser (linéaire)

Forme générale

min ou max $c^T x$
s.c. contraintes $\leq, \geq$ ou $=$
variables ≥ 0 ou libres

*La forme naturelle,
obtenue directement avec
la méthode en 4 étapes —
sans se soucier du format*

Forme canonique

maximiser $c^T x$
s.c. $Ax \leq b$
 $x \geq 0$

*Format homogène
nécessaire pour construire
le dual (Partie 2) et
comparer des PL entre eux*

Forme standard

maximiser $c^T x$
s.c. $Ax = b$
 $x \geq 0$

*La forme qu'exige le
simplexe : il lui faut des
égalités pour construire le
tableau et démarrer avec
une solution de base
immédiate*

- **Générale** → **canonique** : un objectif min devient $\max(-f)$; une contrainte $\geq$ se multiplie par -1 ; une égalité devient deux inégalités $\leq$ (une, et sa version $\times(-1)$) ; une variable libre x devient $x^+ - x^-$, avec $x^+, x^- \geq 0$.
- **Canonique** → **standard** : chaque contrainte $\leq$ reçoit sa propre variable d'écart $s \geq 0$, qui absorbe l'inégalité : $a^T x \leq b$ devient $a^T x + s = b$

Zoom : les trois formes sur un exemple

Modéliser (linéaire)

① Générale

$$\text{minimiser } z = 2x_1 - 3x_2$$

$$\text{s.c. } x_1 + x_2 \geq 4$$

$$2x_1 - x_2 = 6$$

$$x_1 \geq 0, x_2 \text{ libre } (\in \mathbb{R})$$

Zoom : les trois formes sur un exemple

Modéliser (linéaire)

① Générale

minimiser $z = 2x_1 - 3x_2$

s.c. $x_1 + x_2 \geq 4$

$2x_1 - x_2 = 6$

$x_1 \geq 0, x_2 \text{ libre } (\in \mathbb{R})$

- $\min \rightarrow \max(-f)$
- $x_1 + x_2 \geq 4 \rightarrow \times(-1)$
- $2x_1 - x_2 = 6 \rightarrow \text{deux } \leq$
- $x_2 \text{ libre} \rightarrow x_2^+ - x_2^- \text{ } (x_2^+, x_2^- \geq 0)$

Zoom : les trois formes sur un exemple

Modéliser (linéaire)

① Générale

minimiser $z = 2x_1 - 3x_2$

s.c. $x_1 + x_2 \geq 4$

$2x_1 - x_2 = 6$

$x_1 \geq 0, x_2 \text{ libre } (\in \mathbb{R})$

- $\min \rightarrow \max(-f)$
- $x_1 + x_2 \geq 4 \rightarrow \times(-1)$
- $2x_1 - x_2 = 6 \rightarrow \text{deux } \leq$
- $x_2 \text{ libre} \rightarrow x_2^+ - x_2^- \ (x_2^+, x_2^- \geq 0)$

② Canonique

maximiser $w = -2x_1 + 3x_2^+ - 3x_2^- \ (w = -z)$

s.c. $-x_1 - x_2^+ + x_2^- \leq -4$

$2x_1 - x_2^+ + x_2^- \leq 6$

$-2x_1 + x_2^+ - x_2^- \leq -6$

$x_1, x_2^+, x_2^- \geq 0$

Zoom : les trois formes sur un exemple

Modéliser (linéaire)

① Générale

minimiser $z = 2x_1 - 3x_2$

s.c. $x_1 + x_2 \geq 4$

$2x_1 - x_2 = 6$

$x_1 \geq 0, x_2 \text{ libre } (\in \mathbb{R})$

- $\min \rightarrow \max(-f)$
- $x_1 + x_2 \geq 4 \rightarrow \times(-1)$
- $2x_1 - x_2 = 6 \rightarrow \text{deux } \leq$
- $x_2 \text{ libre} \rightarrow x_2^+ - x_2^- \ (x_2^+, x_2^- \geq 0)$

② Canonique

maximiser $w = -2x_1 + 3x_2^+ - 3x_2^- \ (w = -z)$

s.c. $-x_1 - x_2^+ + x_2^- \leq -4$

$2x_1 - x_2^+ + x_2^- \leq 6$

$-2x_1 + x_2^+ - x_2^- \leq -6$

$x_1, x_2^+, x_2^- \geq 0$

- chaque contrainte $\leq$ reçoit une variable d'écart : $+s_1, +s_2, +s_3 \ (s_1, s_2, s_3 \geq 0)$, qui devient une égalité

Zoom : les trois formes sur un exemple

Modéliser (linéaire)

① Générale

minimiser $z = 2x_1 - 3x_2$

s.c. $x_1 + x_2 \geq 4$

$2x_1 - x_2 = 6$

$x_1 \geq 0, x_2 \text{ libre } (\in \mathbb{R})$

- $\min \rightarrow \max(-f)$
- $x_1 + x_2 \geq 4 \rightarrow \times(-1)$
- $2x_1 - x_2 = 6 \rightarrow \text{deux } \leq$
- $x_2 \text{ libre} \rightarrow x_2^+ - x_2^- \ (x_2^+, x_2^- \geq 0)$

② Canonique

maximiser $w = -2x_1 + 3x_2^+ - 3x_2^- \ (w = -z)$

s.c. $-x_1 - x_2^+ + x_2^- \leq -4$

$2x_1 - x_2^+ + x_2^- \leq 6$

$-2x_1 + x_2^+ - x_2^- \leq -6$

$x_1, x_2^+, x_2^- \geq 0$

- chaque contrainte $\leq$ reçoit une variable d'écart : $+s_1, +s_2, +s_3 \ (s_1, s_2, s_3 \geq 0)$, qui devient une égalité

③ Standard

maximiser $w = -2x_1 + 3x_2^+ - 3x_2^-$

s.c. $-x_1 - x_2^+ + x_2^- + s_1 = -4$

$2x_1 - x_2^+ + x_2^- + s_2 = 6$

$-2x_1 + x_2^+ - x_2^- + s_3 = -6$

$x_1, x_2^+, x_2^-, s_1, s_2, s_3 \geq 0$

Modélisation — les points clés

Récap partie 1

- Modéliser, c'est répondre à trois questions : variables, objectif, contraintes
- Un problème continu (variables réelles) se modélise souvent comme un ajustement ou un réglage fin, sans contraintes complexes
- Un problème linéaire se modélise avec la méthode en 4 étapes : variables → objectif → contraintes → domaine
- **La forme du problème (convexe ? linéaire ? continu ou discret ?) détermine déjà, avant toute résolution, s'il sera facile ou difficile**

2

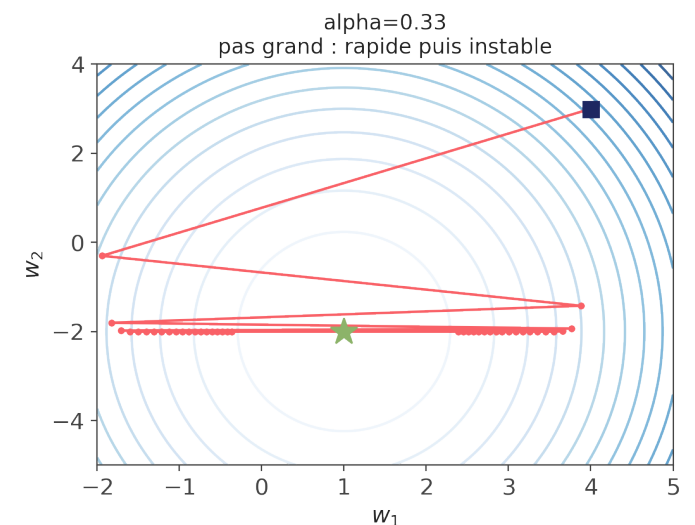
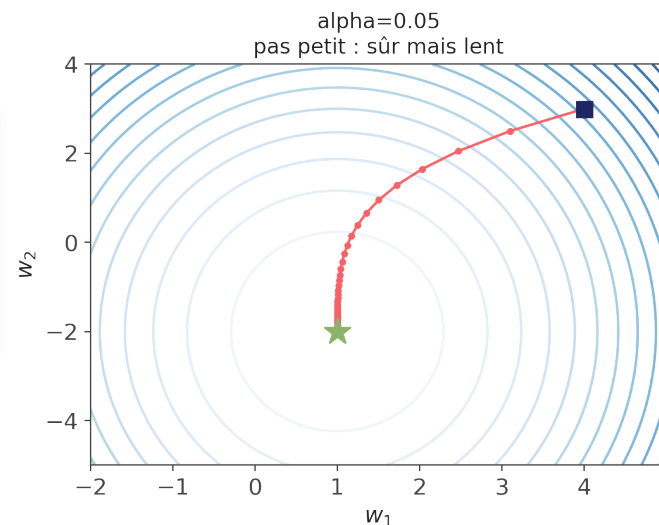
Résolution

Une fois le problème bien posé, comment trouve-t-on effectivement x^* ?
Deux grandes méthodes aujourd'hui : la descente de gradient, et
l'algorithme du simplexe

L'idée : la descente de gradient

Résoudre (continu)

$$w_{k+1} = w_k - \alpha \nabla J(w_k)$$



- On se déplace à l'opposé du gradient : la direction de plus forte pente, comme dans l'analogie de la vallée
- α (le "pas") contrôle l'amplitude de chaque déplacement
- On répète jusqu'à ce que le gradient soit (presque) nul : on ne descend plus
- Cette méthode marche pour n'importe quelle fonction lisse — c'est pour cela qu'elle est si utilisée, y compris pour entraîner un réseau de neurones

À retenir avant l'exemple à la main

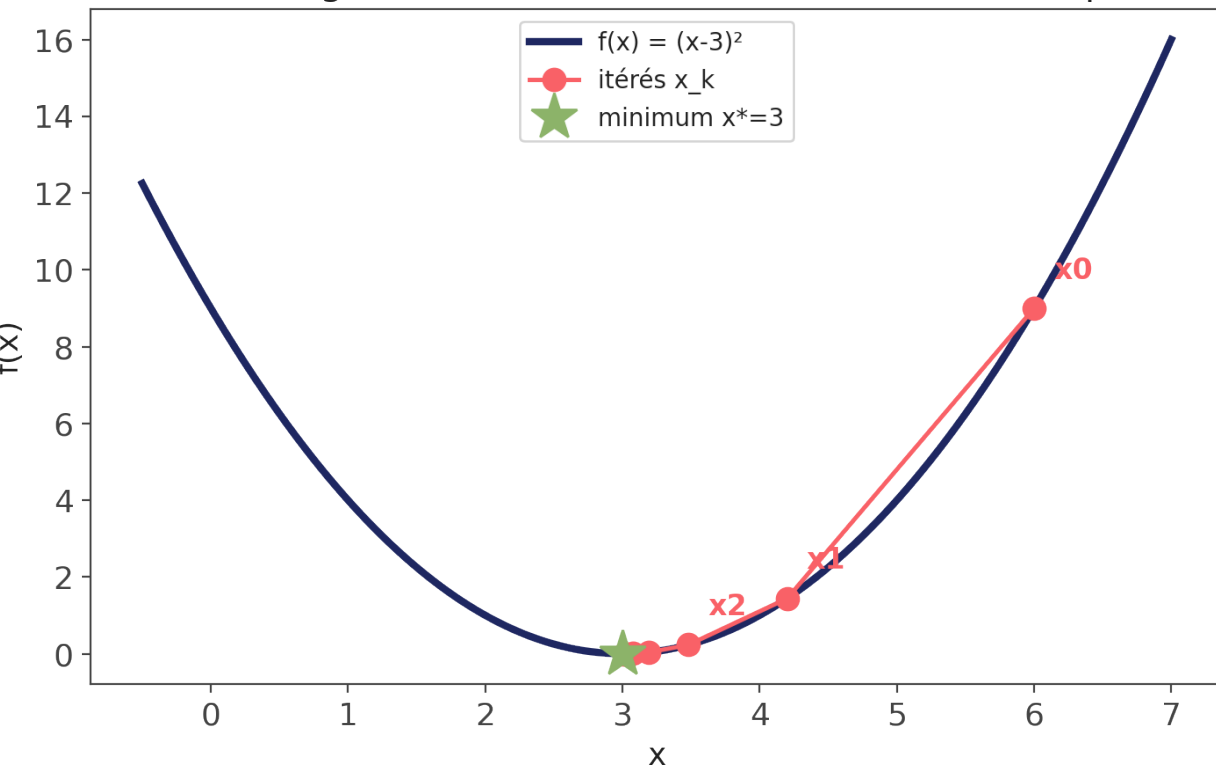
- Le gradient d'une fonction à une variable est simplement sa dérivée
- "Avancer contre le gradient" = "aller vers x plus petit si la pente est positive"
- Page suivante : on calcule tout, à la main, sur un exemple minuscule

Descente de gradient à la main

Exemple minimal: $f(x) = (x - 3)^2$

- $f'(x) = 2(x - 3)$: la règle de mise à jour est : $x_{k+1} = x_k - \alpha 2(x_k - 3)$
- On part de $x_0 = 6$, avec un pas $\alpha = 0.3$
- **Chaque itération réduit l'écart au minimum $x^* = 3$ d'un facteur constant : ici $(1 - 2\alpha)^2 = 0.16$**

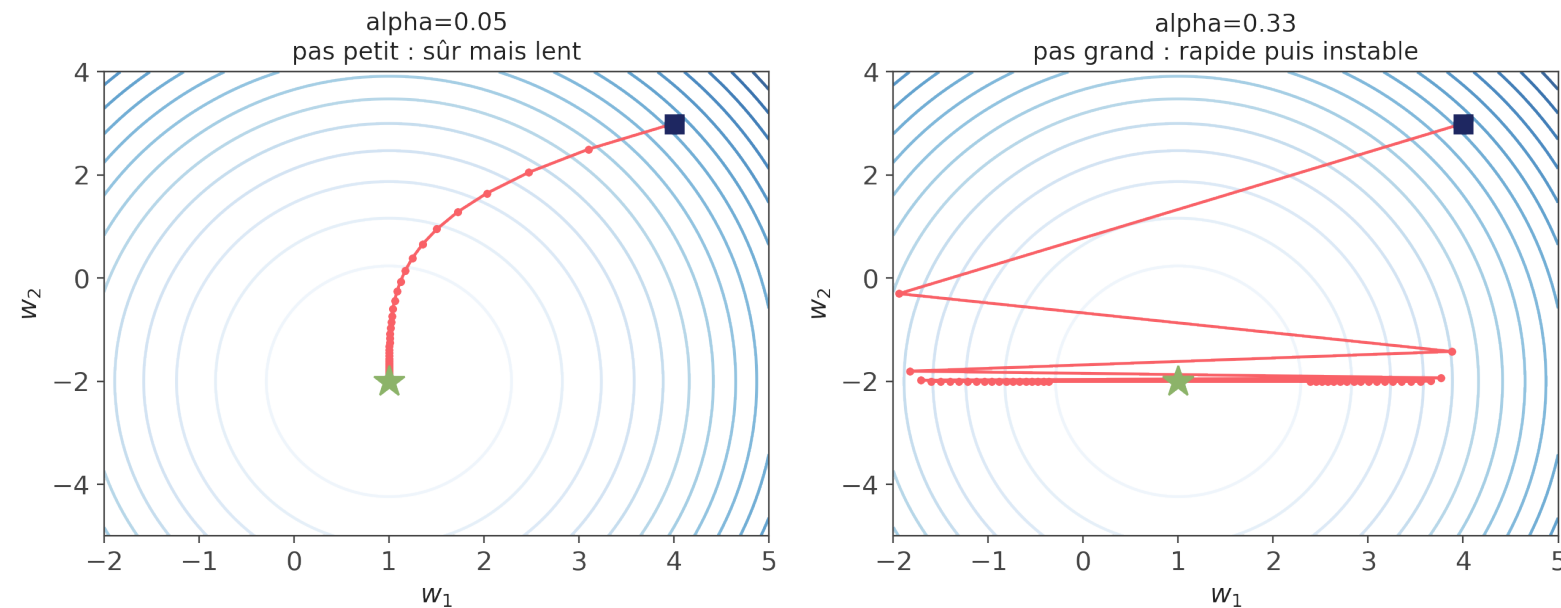
Descente de gradient à la main : $f(x) = (x-3)^2$, $x_0=6$, pas=0.3



k	x_k	$f(x_k)$
0	6.000	9.000
1	4.200	1.440
2	3.480	0.230
3	3.192	0.037
4	3.077	0.006
5	3.031	0.001

Un réglage qui compte

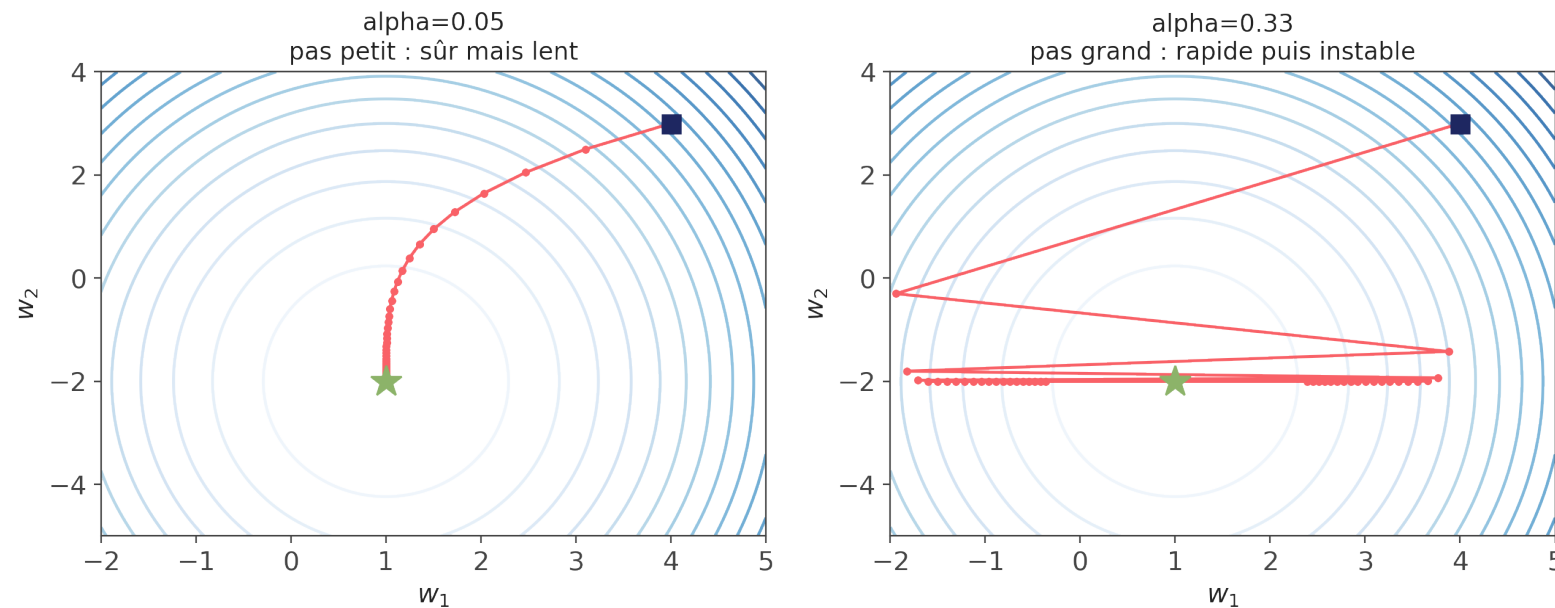
Choisir le pas — et accélérer



- **Trop petit** : convergence sûre, mais douloureusement lente
- **Trop grand** : peut zigzaguer, voire diverger complètement
- Sur un problème mal conditionné (une direction "raide", une autre "plate"), aucun pas fixe n'est vraiment satisfaisant

Un réglage qui compte

Choisir le pas — et accélérer



- **Trop petit** : convergence sûre, mais douloureusement lente
- **Trop grand** : peut zigzaguer, voire diverger complètement
- Sur un problème mal conditionné (une direction "raide", une autre "plate"), aucun pas fixe n'est vraiment satisfaisant

Accélérer : momentum & Nesterov

- **Momentum** : on accumule une "vitesse" au fil des pas, comme une bille qui prend de l'élan
- **Nesterov** : on regarde un peu en avance avant de corriger la trajectoire
- Sur un cas mal conditionné, les deux réduisent nettement le nombre d'itérations nécessaires
- Même principe que celui utilisé, par exemple, par les optimiseurs des réseaux de neurones — une application parmi d'autres

Ajouter des contraintes

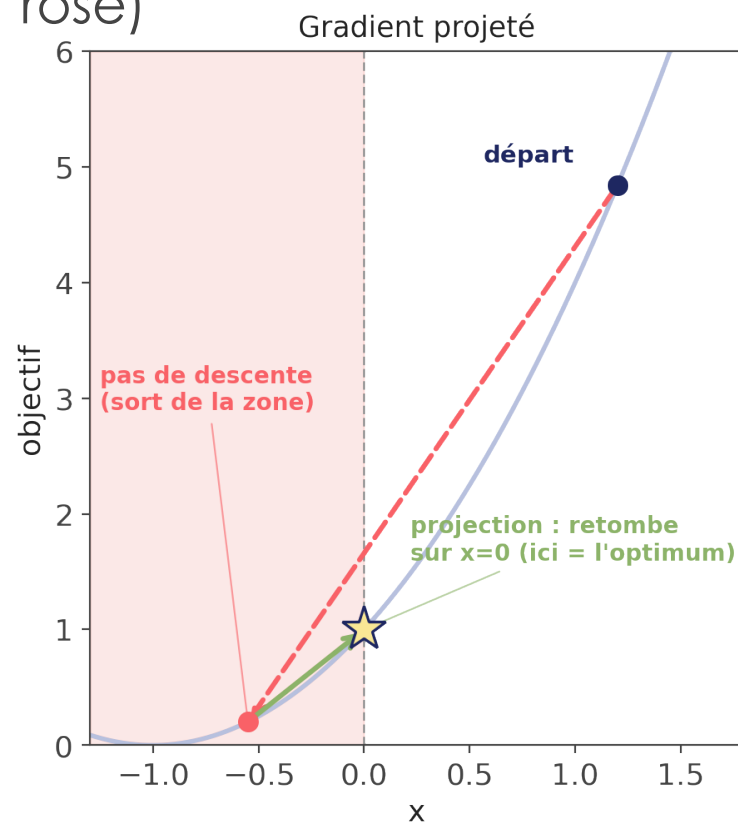
Trois façons de respecter les contraintes

- Un même problème-jouet pour comparer : minimiser $(x + 1)^2$ sous $x \geq 0$. Le minimum sans contrainte ($x = -1$) est interdit ; le vrai optimum contraint est en $x = 0$ (★, zone infaisable en rose)

Ajouter des contraintes

Trois façons de respecter les contraintes

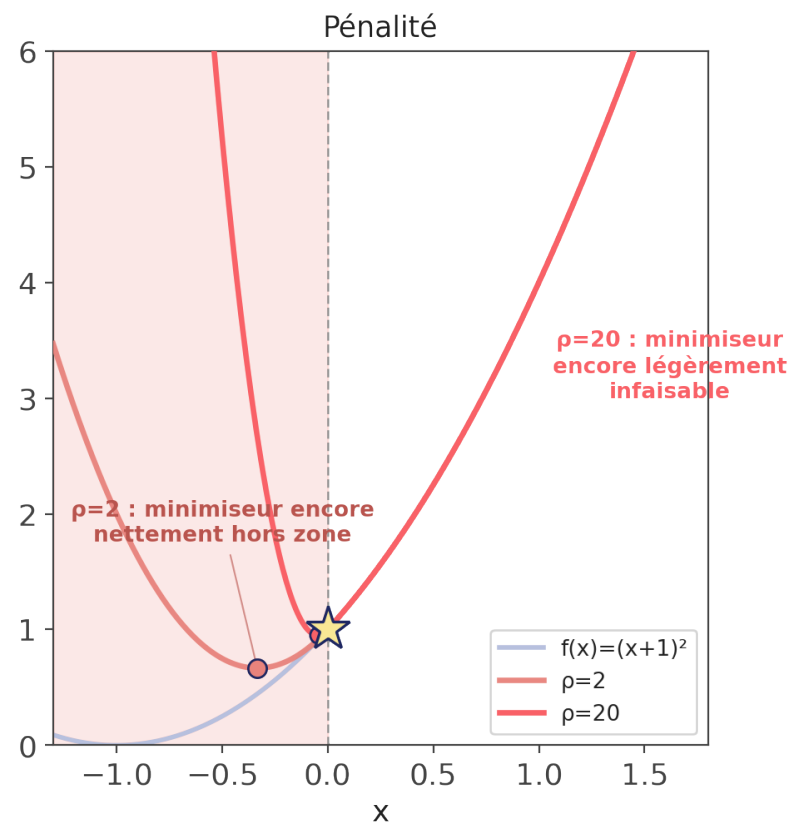
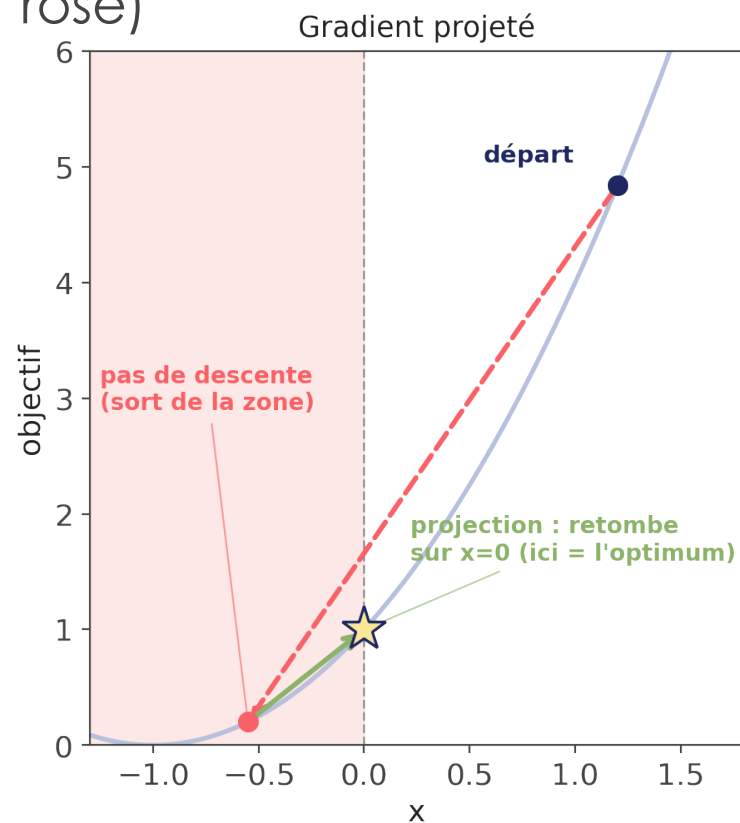
- Un même problème-jouet pour comparer : minimiser $(x + 1)^2$ sous $x \geq 0$. Le minimum sans contrainte ($x = -1$) est interdit ; le vrai optimum contraint est en $x = 0$ (★, zone infaisable en rose)



Ajouter des contraintes

Trois façons de respecter les contraintes

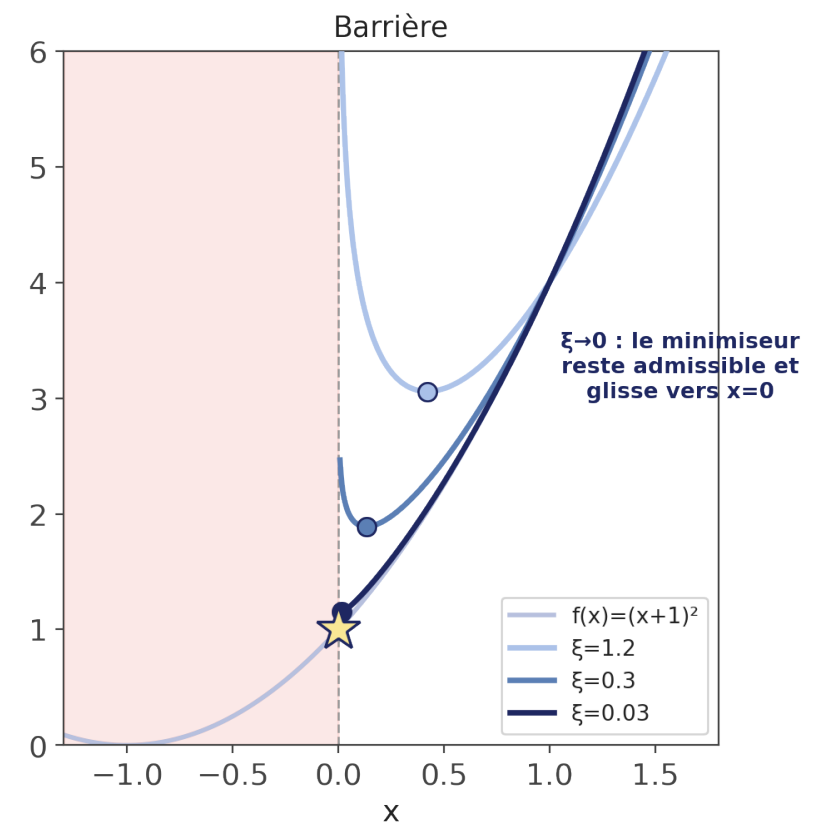
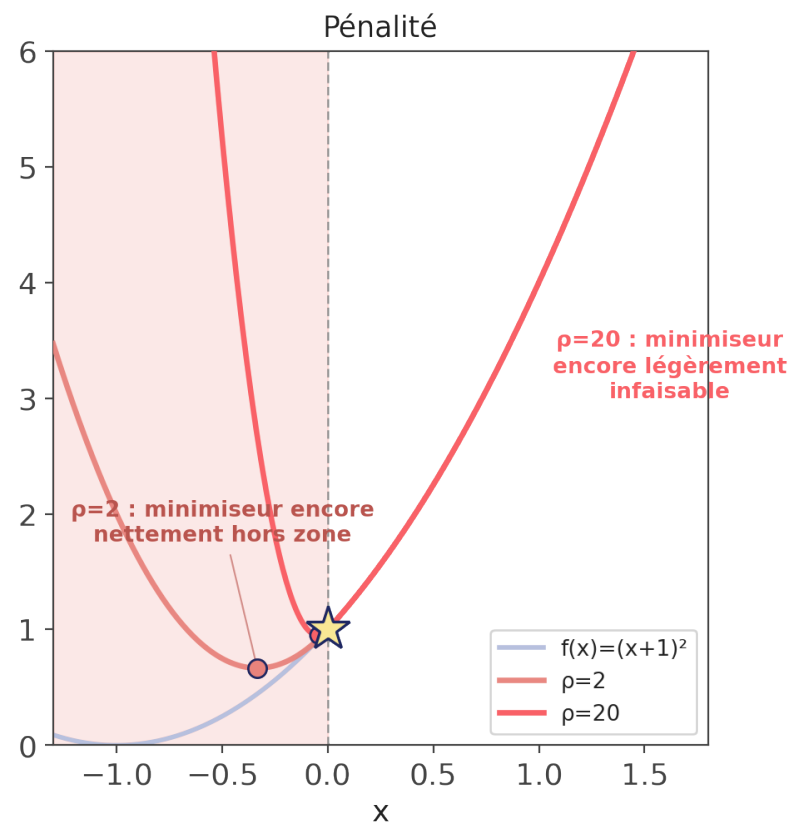
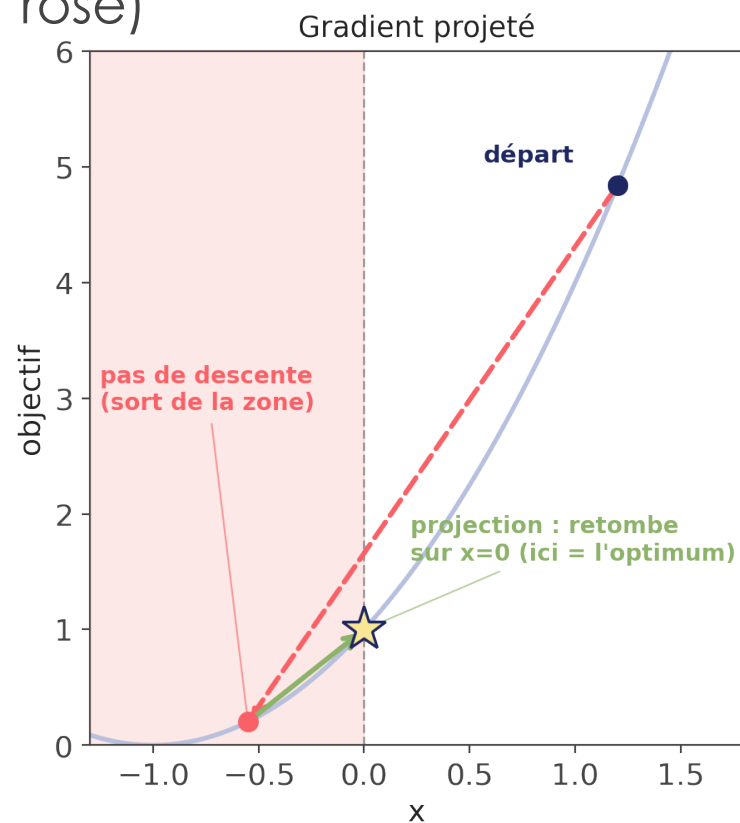
- Un même problème-jouet pour comparer : minimiser $(x + 1)^2$ sous $x \geq 0$. Le minimum sans contrainte ($x = -1$) est interdit ; le vrai optimum contraint est en $x = 0$ (★, zone infaisable en rose)



Ajouter des contraintes

Trois façons de respecter les contraintes

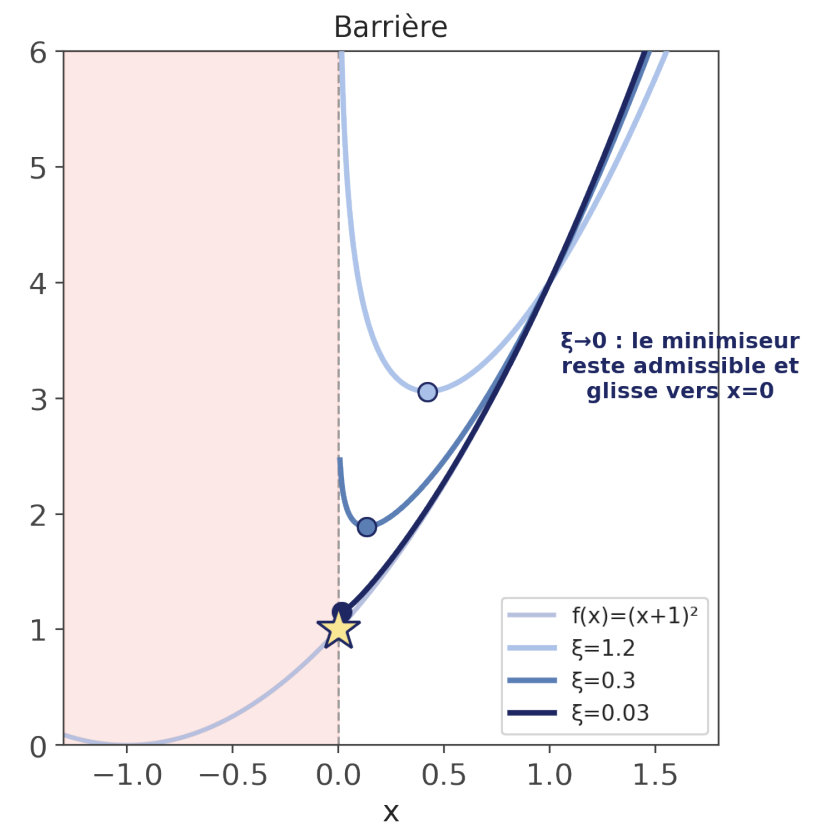
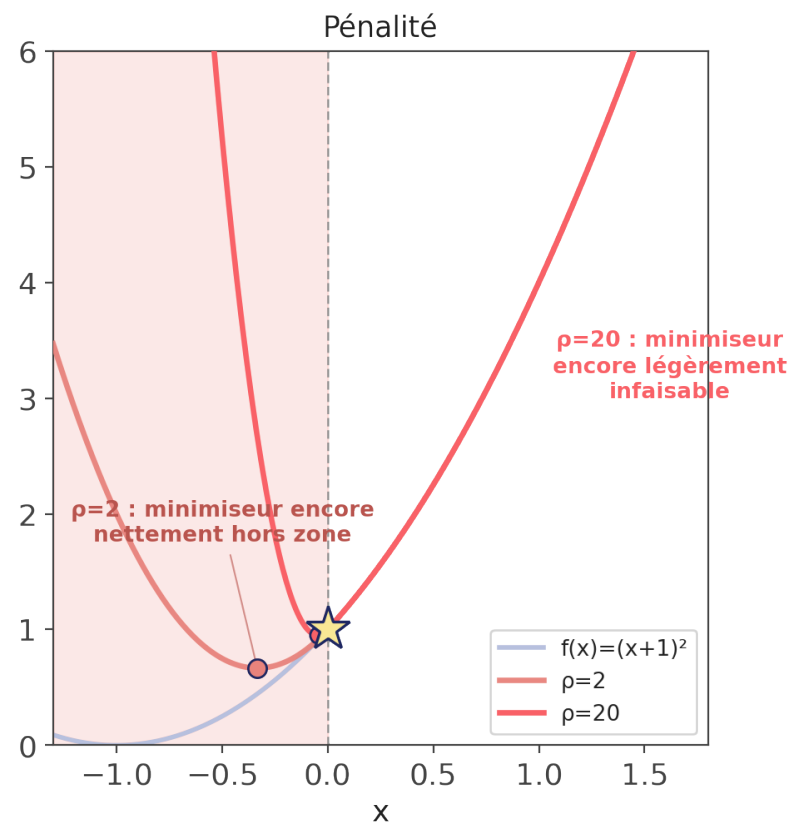
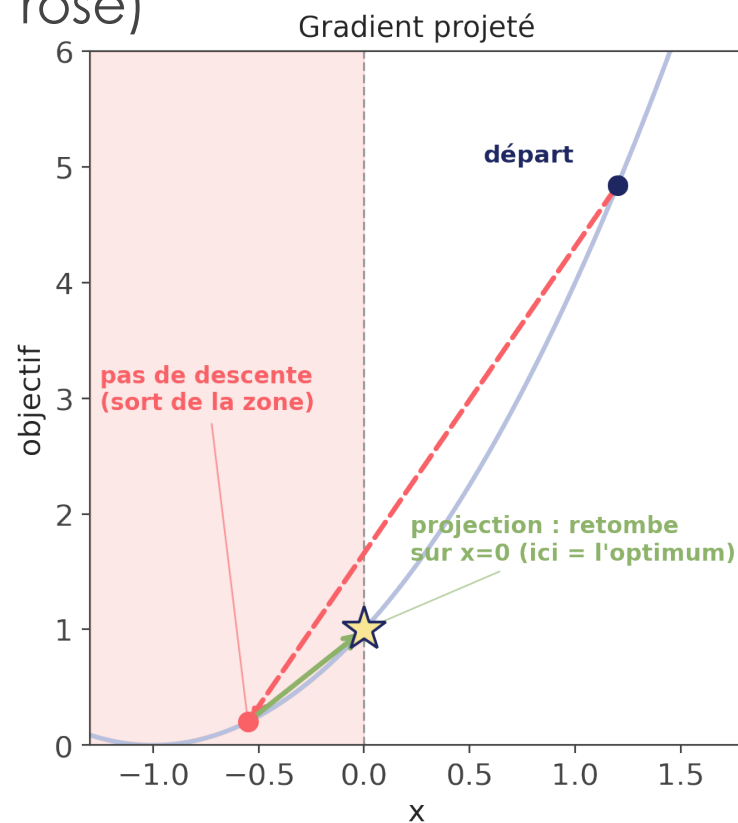
- Un même problème-jouet pour comparer : minimiser $(x + 1)^2$ sous $x \geq 0$. Le minimum sans contrainte ($x = -1$) est interdit ; le vrai optimum contraint est en $x = 0$ (★, zone infaisable en rose)



Ajouter des contraintes

Trois façons de respecter les contraintes

- Un même problème-jouet pour comparer : minimiser $(x + 1)^2$ sous $x \geq 0$. Le minimum sans contrainte ($x = -1$) est interdit ; le vrai optimum contraint est en $x = 0$ (★, zone infaisable en rose)

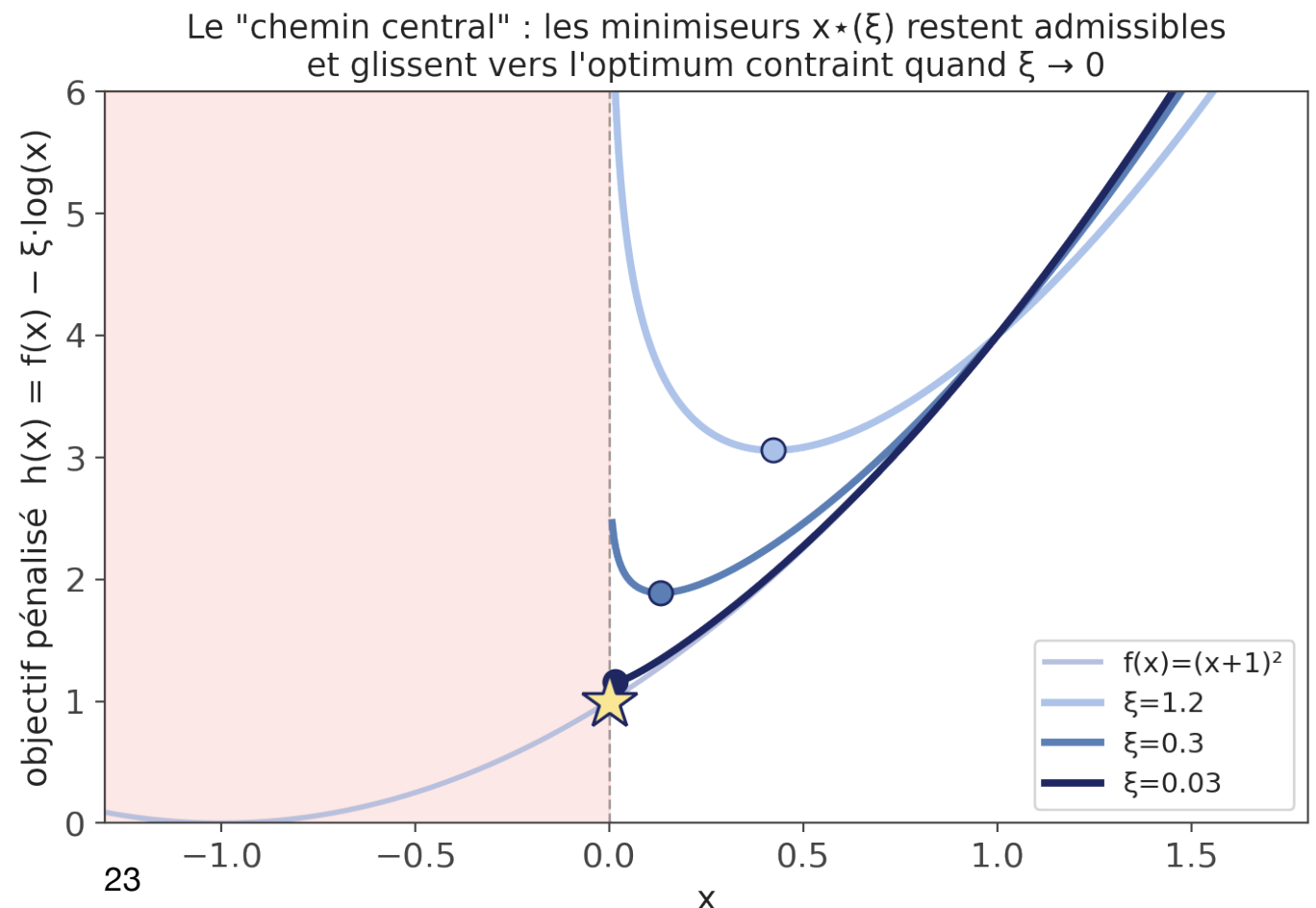


- **Gradient projeté** : simple, mais suppose une projection facile à calculer
- **Pénalité** : facile à coder, mais reste légèrement hors zone tant que ρ est fini
- **Barrière** : plus coûteuse, mais toujours strictement admissible

Ajouter des contraintes

Zoom : la barrière et le "chemin central"

- $h(x) = f(x) - \epsilon \log(x)$: le terme $-\epsilon \log(x) \rightarrow +\infty$ quand $x \rightarrow 0^+$, donc h ne peut jamais être minimisé en sortant de $x > 0$ — chaque itéré reste strictement admissible
- On résout une suite de problèmes SANS contrainte, pour ϵ décroissant (ex. $1,2 \rightarrow 0,3 \rightarrow 0,03 \rightarrow \dots \rightarrow 0$) : la suite des minimiseurs $x^*(\epsilon)$ trace le "chemin central", tracé ci-contre
- **Contraste avec la pénalité : la barrière reste toujours admissible ; la pénalité, elle, ne s'approche de l'admissibilité qu'à la limite $p \rightarrow \infty$**



Lagrange / Karush-Kuhn-Tucker (KKT)

UNE IDÉE PLUS FINE

- À l'optimum contraint, on ne peut plus se contenter d'un gradient nul : il faut tenir compte des contraintes actives

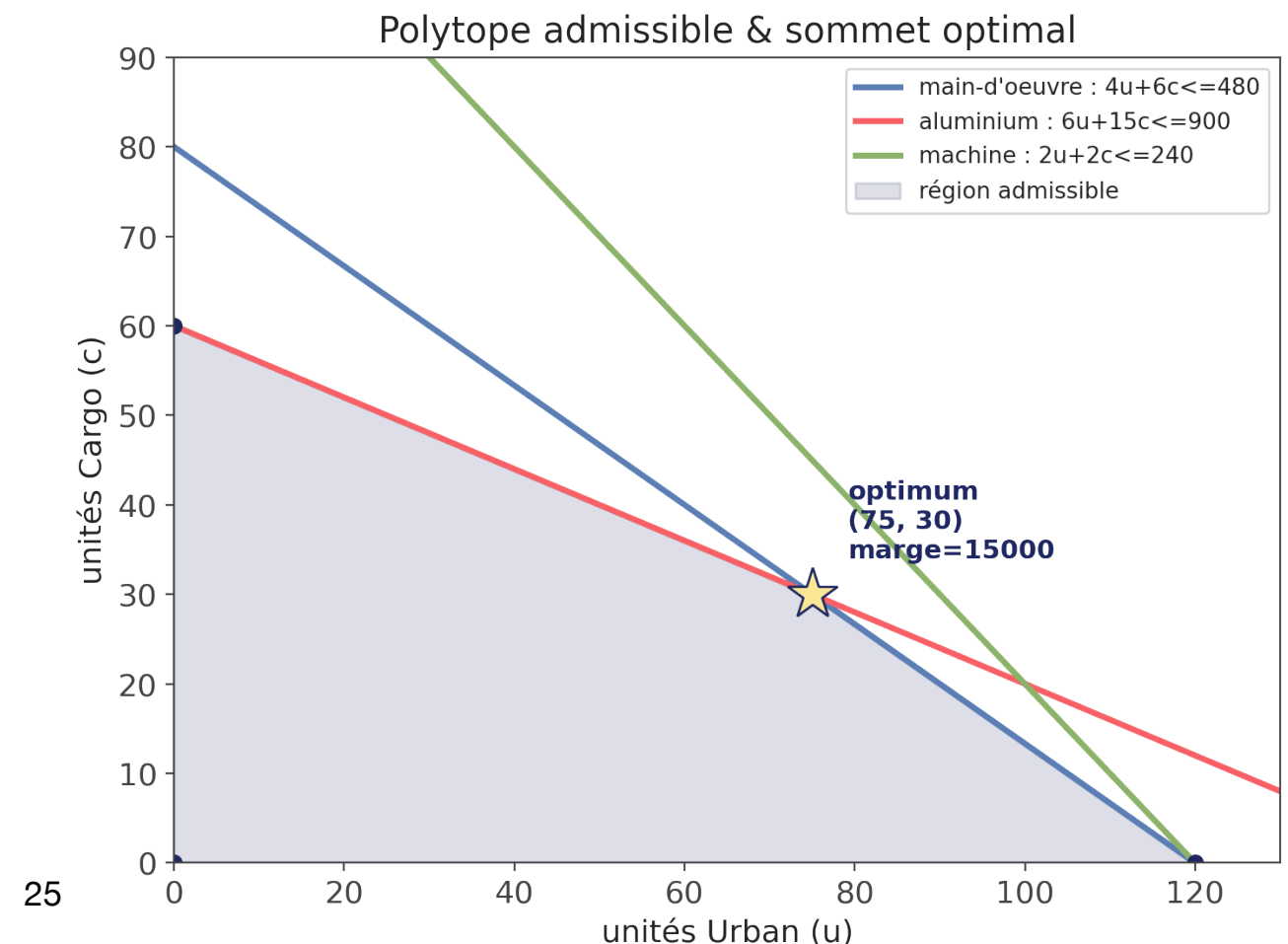
$$\nabla f_0(x^*) = \sum_i \mu_i \nabla g_i(x^*), \quad \mu_i \leq 0, \quad \mu_i g_i(x^*) = 0$$

- **Stationnarité** : À l'optimum, le gradient du coût est une combinaison des gradients des contraintes actives : impossible d'améliorer f_0 sans quitter l'ensemble admissible
- **Signe des multiplicateurs** : $\mu_i \geq 0$: une contrainte ne peut que "repousser" la solution, jamais « l'attirer »
- **Complémentarité** : $\mu_i = 0$ sauf si la contrainte i est active (saturée). Une contrainte non saturée n'influence pas la solution localement

La région admissible est un polytope

Résoudre (linéaire)

- Chaque contrainte linéaire est un demi-espace, donc convexe ; leur intersection reste convexe (propriété vue en Partie 1) — c'est un polytope convexe
- **Un objectif linéaire est toujours optimisé en un sommet du polytope (jamais strictement à l'intérieur)**
- C'est pourquoi le PL est plus facile que l'optimisation continue générale : chercher parmi un nombre fini de sommets plutôt que dans un continuum
- **ExedBike** : le mix optimal hebdomadaire (75 Urban, 30 Cargo) est exactement le sommet où main-d'œuvre et aluminium sont simultanément saturés



L'algorithme du simplexe : principe

L'algorithme

L'algorithme du simplexe : principe

L'algorithme

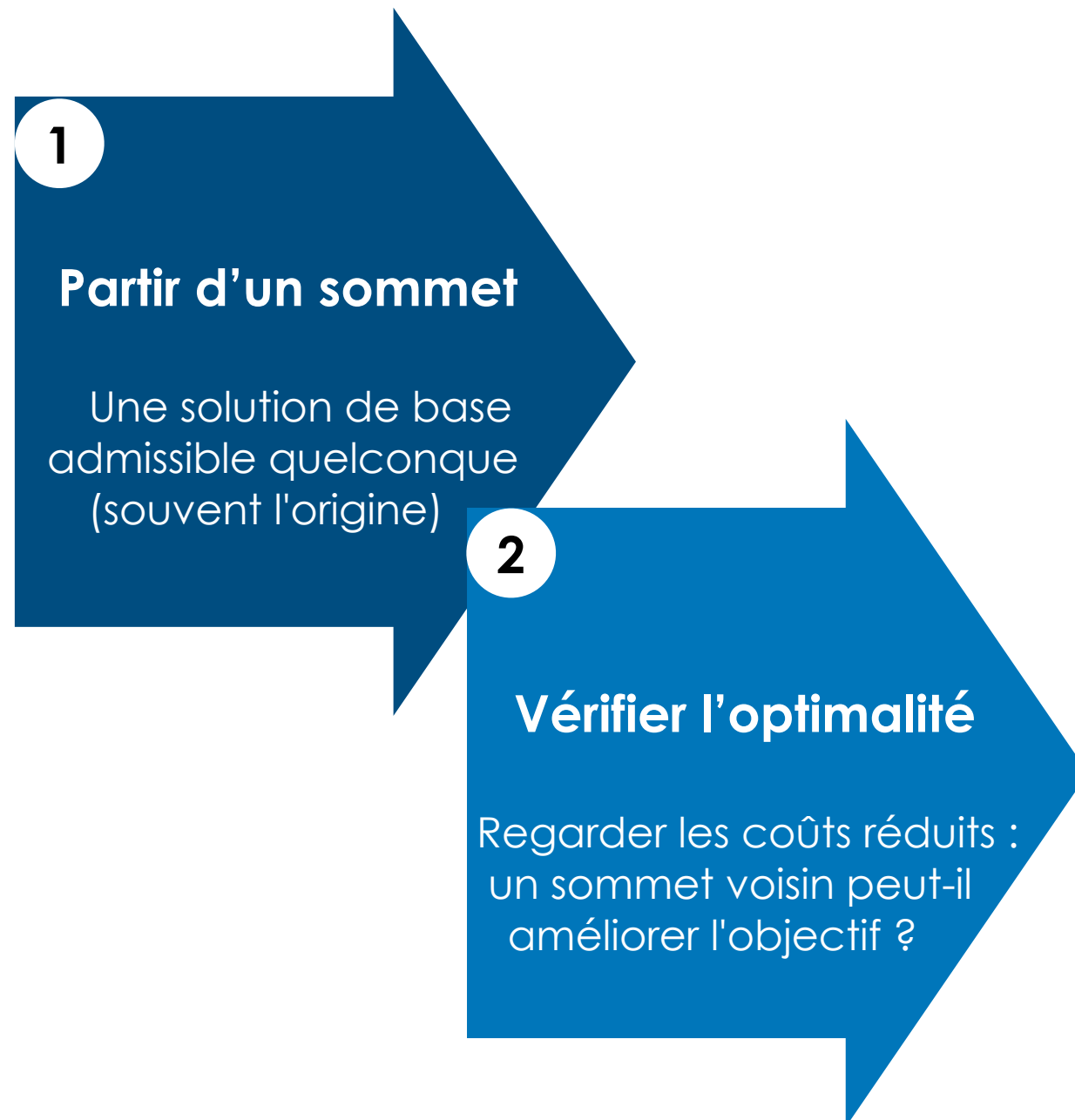
1

Partir d'un sommet

Une solution de base
admissible quelconque
(souvent l'origine)

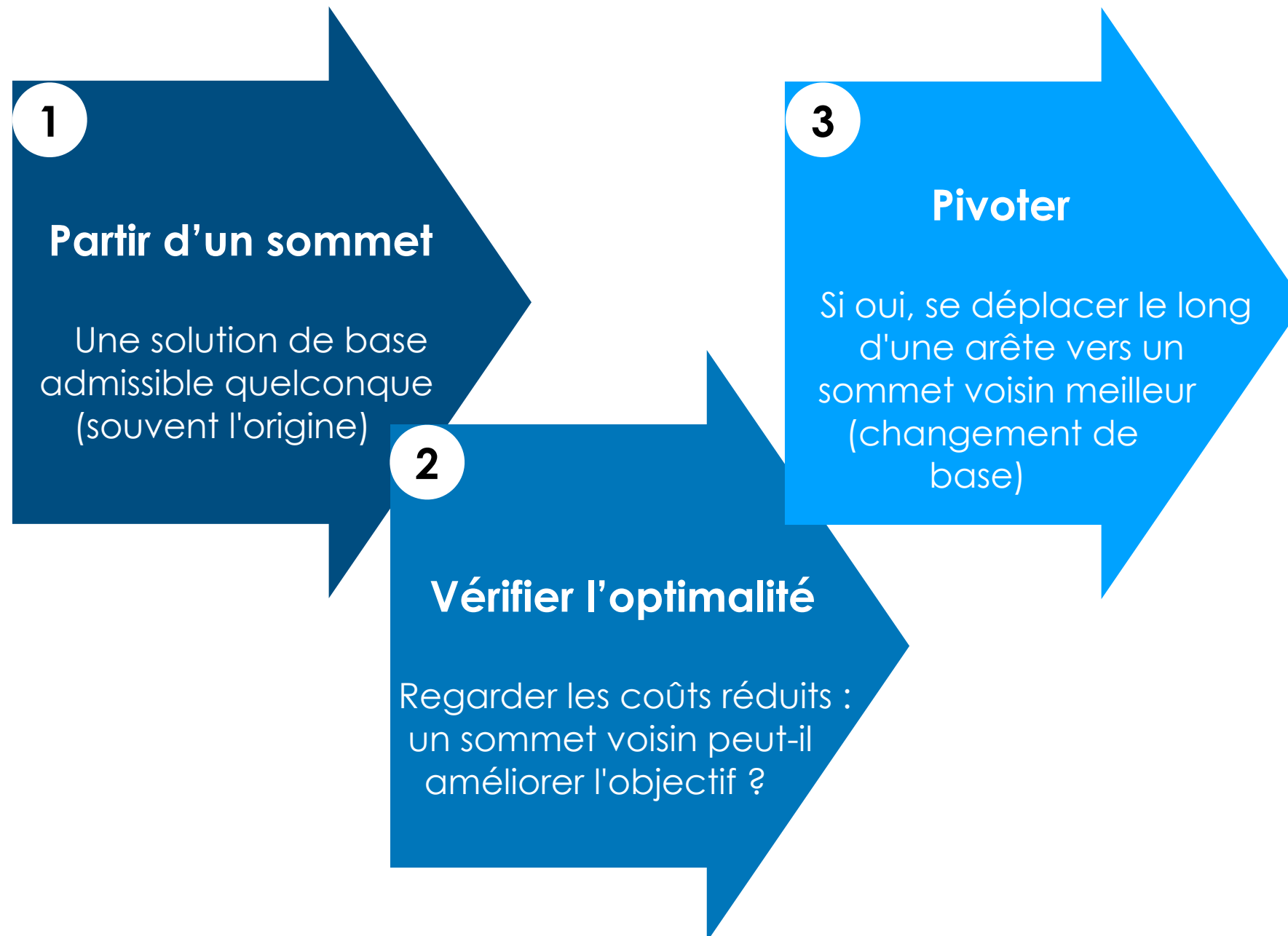
L'algorithme du simplexe : principe

L'algorithme



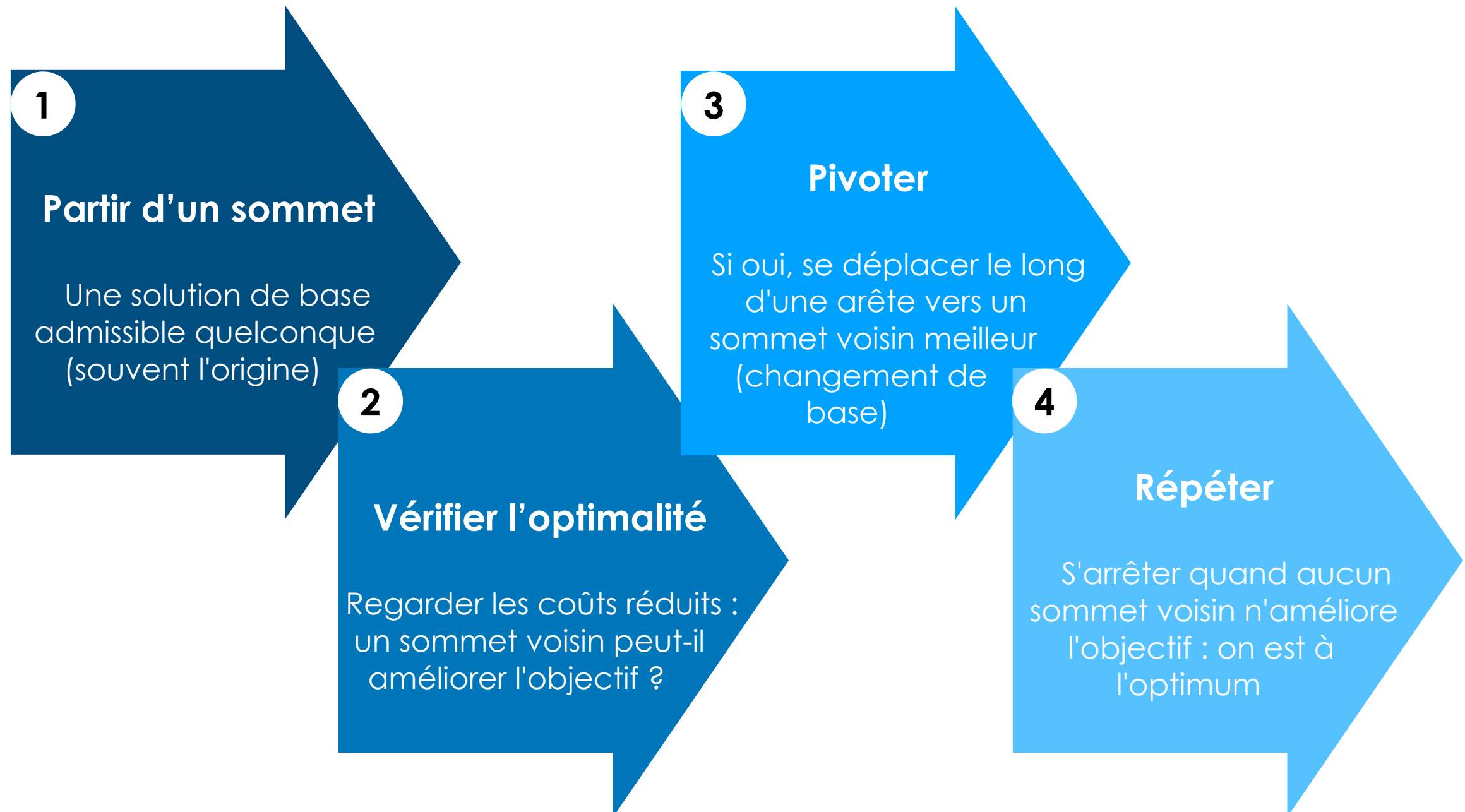
L'algorithme du simplexe : principe

L'algorithme



L'algorithme du simplexe : principe

L'algorithme



Du système standard au tableau initial

L'algorithme

① Système à résoudre

maximiser $70t + 50c$

s.c. $4t + 3c \leq 240$ (bois)

$2t + c \leq 100$ (machine)

$t, c \geq 0$

Du système standard au tableau initial

L'algorithme

① Système à résoudre

maximiser $70t + 50c$

s.c. $4t + 3c \leq 240$ (bois)

$2t + c \leq 100$ (machine)

$t, c \geq 0$

② Forme standard

maximiser $70t + 50c$

s.c. $4t + 3c + s_1 = 240$

$2t + c + s_2 = 100$

$t, c, s_1, s_2 \geq 0$

Du système standard au tableau initial

L'algorithme

① Système à résoudre

maximiser $70t + 50c$

s.c. $4t + 3c \leq 240$ (bois)

$2t + c \leq 100$ (machine)

$t, c \geq 0$

② Forme standard

maximiser $70t + 50c$

s.c. $4t + 3c + s_1 = 240$

$2t + c + s_2 = 100$

$t, c, s_1, s_2 \geq 0$

① Tableau initial

Base	t	c	s ₁	s ₂	RHS
s1	4	3	1	0	240
s2	2	1	0	1	100
z	-70	-50	0	0	0

- Entrante : t (coeff. le plus négatif, -70)
- Rapports : $240/4=60$, $100/2=50 \rightarrow \min$ en s_2
- **Sortante : s_2 . Pivot = 2**

Du système standard au tableau initial

L'algorithme

① Système à résoudre

maximiser $70t + 50c$

s.c. $4t + 3c \leq 240$ (bois)

$2t + c \leq 100$ (machine)

$t, c \geq 0$

② Forme standard

maximiser $70t + 50c$

s.c. $4t + 3c + s_1 = 240$

$2t + c + s_2 = 100$

$t, c, s_1, s_2 \geq 0$

① Tableau initial

Base	t	c	s ₁	s ₂	RHS
s1	4	3	1	0	240
s2	2	1	0	1	100
z	-70	-50	0	0	0

- Entrante : t (coeff. le plus négatif, -70)
- Rapports : $240/4=60$, $100/2=50 \rightarrow \min$ en s_2
- **Sortante : s_2 . Pivot = 2**

② Après pivot 1 (t entre)

Base	t	c	s ₁	s ₂	RHS
s1	0	1	1	-2	40
t	1	0,5	0	0,5	50
z	0	-15	0	35	3 500

- Entrante : c (coeff. encore négatif, -15)
- Rapports : $40/1=40$, $50/0,5=100 \rightarrow \min$ en s_1
- **Sortante : s_1 . Pivot = 1**

Du système standard au tableau initial

L'algorithme

① Système à résoudre

maximiser $70t + 50c$

s.c. $4t + 3c \leq 240$ (bois)

$2t + c \leq 100$ (machine)

$t, c \geq 0$

② Forme standard

maximiser $70t + 50c$

s.c. $4t + 3c + s_1 = 240$

$2t + c + s_2 = 100$

$t, c, s_1, s_2 \geq 0$

① Tableau initial

Base	t	c	s ₁	s ₂	RHS
s1	4	3	1	0	240
s2	2	1	0	1	100
Z	-70	-50	0	0	0

- Entrante : t (coeff. le plus négatif, -70)
- Rapports : $240/4=60$, $100/2=50 \rightarrow \min$ en s_2
- **Sortante : s_2 . Pivot = 2**

② Après pivot 1 (t entre)

Base	t	c	s ₁	s ₂	RHS
s1	0	1	1	-2	40
t	1	0,5	0	0,5	50
Z	0	-15	0	35	3 500

- Entrante : c (coeff. encore négatif, -15)
- Rapports : $40/1=40$, $50/0,5=100 \rightarrow \min$ en s_1
- **Sortante : s_1 . Pivot = 1**

③ Après pivot 2 — optimal

Base	t	c	s ₁	s ₂	RHS
c	0	1	1	-2	40
t	1	0	-0,5	1,5	30
Z	0	0	15	5	4 100

- **Ligne Z entièrement $\geq 0 \rightarrow$ optimum atteint**
- $t=30$, $c=40$, $Z=4\,100$ € — identique à la lecture par sommets

Résoudre à la main

la méthode des sommets

① Système à résoudre

maximiser $70t + 50c$

s.c. $4t + 3c \leq 240$ (bois)

$2t + c \leq 100$ (machine)

$t, c \geq 0$

- Sur un problème à 2 variables, pas besoin de tableau : le polytope admissible a 4 sommets, et c'est là, et seulement là, que se trouve l'optimum
- Il suffit d'évaluer l'objectif en chacun, et de garder le plus grand
- Même résultat que le simplexe — juste une autre façon d'y arriver

Résoudre à la main

la méthode des sommets

① Système à résoudre

maximiser $70t + 50c$

s.c. $4t + 3c \leq 240$ (bois)

$2t + c \leq 100$ (machine)

$t, c \geq 0$

Sommet (t, c)	Objectif $70t+50c$
(0, 0)	0 €
(0, 80)	4 000 €
(30, 40)	4 100 € ★ optimum
(50, 0)	3 500 €

Optimum : $t = 30, c = 40$

Marge = $70 \times 30 + 50 \times 40 = 4\,100$ €

- Sur un problème à 2 variables, pas besoin de tableau : le polytope admissible a 4 sommets, et c'est là, et seulement là, que se trouve l'optimum
- Il suffit d'évaluer l'objectif en chacun, et de garder le plus grand
- Même résultat que le simplexe — juste une autre façon d'y arriver

Résolution — les points clés

Récap partie 2

- Continu : la descente de gradient suit la pente, un petit pas à la fois ; momentum/Nesterov l'accélèrent
- Les contraintes se gèrent par projection, pénalité ou barrière ; Lagrange/KKT donnent les conditions d'optimalité exactes
- Linéaire : l'optimum est toujours à un sommet du polytope ; le simplexe (ou la méthode des sommets, à la main) le trouve à coup sûr
- **Dans les deux cas, on trouve un optimum global certifié — c'est la récompense d'avoir un problème convexe ou linéaire**

Optimisation pour l'IA

Cours1 - Optimisation continue et linéaire

MS IA DE CONFIANCE · CENTRALESUPÉLEC

Nadjib LAZAAR

Ing - Phd - HDR - Professor

Paris-Saclay University - LISN - DataIA Institute

<https://perso.lisn.upsaclay.fr/lazaar/>

24/09/2026